

# Sensor-based Fuzzy Control of a Rigid Bodied 4-Legged Robot

by

Adrian Iannantuono

Electrical Engineering Project  
on Intelligent Robot Control  
(M.Eng. Project)

ELG 5901

Supervised by: Dr. Emil Petriu

Ottawa, Ontario, Canada,

© Adrian Iannantuono 2024

## **Abstract**

Locomotion of robots over uneven terrain remains a challenge for the robotic industry. Techniques to solve this problem include emulating biological movements and their ability to adapt to changing environments. Development of such systems are pivotal in the pursuit of useful and reliable robotic systems. These systems can be used in situations such as search and rescue operations in addition to other scenarios which have the ability to mitigate human risk. This paper will present a project focused on addressing the challenge of robotic motion on uneven surfaces by developing a rigid-bodied 4-legged robot which is able to dynamically adjust its foot positioning in order to maintain stability. Integrating contact sensors into the feet of the robot allows an adaptive control system based on fuzzy logic to be used to control the robots next movements. The fuzzy logic system allows for real-time adjustments in next state motor positioning which improves stability and lowers motor fatigue when compared to a fixed programmed design. The proposed solution will be developed using MATLAB's Simulink Multibody tool which will allow for simulations to be run and data to be collected for further analysis. Comparisons between the fixed movement and fuzzy logic designed control systems will be presented through analysis of simulation data.

## **Acknowledgements**

I would like to thank Dr. Emil Petriu for supervising this project throughout the semester.

# Table of Contents

|                                     |          |
|-------------------------------------|----------|
| List of Tables                      | vii      |
| List of Figures                     | viii     |
| Abbreviations                       | xiii     |
| <b>1 Introduction</b>               | <b>1</b> |
| 1.1 Problem Definition . . . . .    | 3        |
| 1.2 Objectives . . . . .            | 4        |
| <b>2 Literature Survey</b>          | <b>6</b> |
| <b>3 Proposed Solution</b>          | <b>8</b> |
| 3.1 Robot Design . . . . .          | 8        |
| 3.2 Robot Movement . . . . .        | 10       |
| 3.3 Simulation Overview . . . . .   | 11       |
| 3.4 Ground Initialisation . . . . . | 12       |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| 3.5      | Body Subsystem . . . . .                | 13        |
| 3.6      | Front and Rear Legs Subsystem . . . . . | 15        |
| 3.6.1    | Leg Actuation . . . . .                 | 17        |
| 3.6.2    | Contact Sensing . . . . .               | 19        |
| 3.7      | Stop Legs Controller . . . . .          | 20        |
| 3.8      | Body Controller . . . . .               | 21        |
| 3.8.1    | Fuzzy Logic System . . . . .            | 22        |
| <b>4</b> | <b>Results</b>                          | <b>28</b> |
| 4.1      | Level Surface . . . . .                 | 28        |
| 4.2      | Object Under Left Legs . . . . .        | 32        |
| 4.3      | Object Under Right Legs . . . . .       | 34        |
| <b>5</b> | <b>Future Work and Conclusion</b>       | <b>37</b> |
| 5.1      | Future Work . . . . .                   | 37        |
| 5.2      | Conclusion . . . . .                    | 38        |
|          | <b>APPENDICES</b>                       | <b>40</b> |
| <b>A</b> | <b>MATLAB Code</b>                      | <b>41</b> |
| <b>B</b> | <b>Results Data</b>                     | <b>44</b> |
| B.1      | Level Surface . . . . .                 | 44        |

|                                       |           |
|---------------------------------------|-----------|
| B.2 Object Under Left Legs . . . . .  | 48        |
| B.3 Object Under Right Legs . . . . . | 52        |
| <b>References</b>                     | <b>56</b> |

# List of Tables

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 3.1 | Body properties. . . . .                              | 14 |
| 3.2 | Fuzzy rules with robot on level surface [1] . . . . . | 25 |
| 3.3 | Fuzzy rules with front leg on object [1] . . . . .    | 26 |
| 3.4 | Fuzzy rules with rear leg on object [1] . . . . .     | 26 |

# List of Figures

|      |                                                                 |    |
|------|-----------------------------------------------------------------|----|
| 1.1  | Fuzzy logic. . . . .                                            | 3  |
| 1.2  | Robot centre of mass in support region. . . . .                 | 4  |
| 3.1  | Robot front view (a) On level ground, (b) In motion. . . . .    | 9  |
| 3.2  | Robot overhead view (a) On level ground, (b) In motion. . . . . | 9  |
| 3.3  | Movement sequence. . . . .                                      | 10 |
| 3.4  | Simulation overview. . . . .                                    | 12 |
| 3.5  | 3D simulation. . . . .                                          | 12 |
| 3.6  | Body subsystem. . . . .                                         | 15 |
| 3.7  | Body geometry. . . . .                                          | 15 |
| 3.8  | Front or rear legs subsystem. . . . .                           | 16 |
| 3.9  | Leg geometry (a) Front view, (b) Perspective view. . . . .      | 17 |
| 3.10 | Front or rear legs subsystem actuation. . . . .                 | 18 |
| 3.11 | Step input and smoothing transfer function output. . . . .      | 18 |
| 3.12 | Leg actuation (a) Side view, (b) Top view. . . . .              | 19 |

|      |                                                                                                                                                                                                                                                                                                                                                   |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.13 | Foot contact sensors. . . . .                                                                                                                                                                                                                                                                                                                     | 20 |
| 3.14 | Stop legs controller subsystem. . . . .                                                                                                                                                                                                                                                                                                           | 21 |
| 3.15 | Body controller subsystem. . . . .                                                                                                                                                                                                                                                                                                                | 23 |
| 3.16 | Membership functions (a) Input angle Alpha1 or Alpha2 ( $\alpha 1$ or $\alpha 2$ ), (b)<br>Input angle Beta ( $\beta$ ), (c) Output angle Alpha1' or Alpha2' ( $\alpha 1'$ or $\alpha 2'$ ). . .                                                                                                                                                  | 25 |
| 3.17 | Control surfaces (a) Alpha1 vs. Beta vs. Alpha1' ( $\alpha 1$ vs. $\beta$ vs. $\alpha 1'$ ), (b)<br>Alpha2 vs. Beta vs. Alpha2' ( $\alpha 2$ vs. $\beta$ vs. $\alpha 2'$ ), (c) Alpha1 vs. Beta vs.<br>Alpha2' ( $\alpha 1$ vs. $\beta$ vs. $\alpha 2'$ ), (d) Alpha2 vs. Beta vs. Alpha1' ( $\alpha 2$ vs. $\beta$ vs.<br>$\alpha 1'$ ). . . . . | 27 |
| 4.1  | Fixed programmed simulation screenshots on level surface (a) time=4s, (b)<br>time=6s, (c) time=8s, (d) time=10s, (e) time=12s, (f) time=14s. . . . .                                                                                                                                                                                              | 30 |
| 4.2  | Fuzzy logic controlled simulation screenshots on level surface (a) time=4s,<br>(b) time=6s, (c) time=8s, (d) time=10s, (e) time=12s, (f) time=14s. . . .                                                                                                                                                                                          | 31 |
| 4.3  | Fixed programmed simulation screenshots with object under left legs (a)<br>time=12s, (b) time=14s, (c) time=16s, (d) time=18s, (e) time=20s, (f)<br>time=22s. . . . .                                                                                                                                                                             | 33 |
| 4.4  | Fuzzy logic controlled simulation screenshots with object under left legs<br>(a) time=12s, (b) time=14s, (c) time=16s, (d) time=18s, (e) time=20s, (f)<br>time=22s. . . . .                                                                                                                                                                       | 34 |
| 4.5  | Fixed programmed simulation screenshots with object under right legs (a)<br>time=18s, (b) time=20s, (c) time=22s, (d) time=24s, (e) time=26s, (f)<br>time=28s. . . . .                                                                                                                                                                            | 35 |

|     |                                                                                                                                                                              |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.6 | Fuzzy logic controlled simulation screenshots with object under right legs<br>(a) time=18s, (b) time=20s, (c) time=22s, (d) time=24s, (e) time=26s, (f)<br>time=28s. . . . . | 36 |
| A.1 | init-ground.m. . . . .                                                                                                                                                       | 41 |
| A.2 | stop-legs-controller.m. . . . .                                                                                                                                              | 42 |
| A.3 | body-controller.m. . . . .                                                                                                                                                   | 43 |
| B.1 | Front legs angle on level surface (a) Fixed programmed control, (b) Fuzzy<br>logic control. . . . .                                                                          | 44 |
| B.2 | Rear legs angle on level surface (a) Fixed programmed control, (b) Fuzzy<br>logic control. . . . .                                                                           | 45 |
| B.3 | Body position on level surface (a) Fixed programmed control, (b) Fuzzy<br>logic control. . . . .                                                                             | 45 |
| B.4 | Body angle on level surface (a) Fixed programmed control, (b) Fuzzy logic<br>control. . . . .                                                                                | 46 |
| B.5 | Body angular acceleration on level surface (a) Fixed programmed control,<br>(b) Fuzzy logic control. . . . .                                                                 | 46 |
| B.6 | Front legs torque on level surface (a) Fixed programmed control, (b) Fuzzy<br>logic control. . . . .                                                                         | 47 |
| B.7 | Rear legs torque on level surface (a) Fixed programmed control, (b) Fuzzy<br>logic control. . . . .                                                                          | 47 |
| B.8 | Front legs angle walking over object on the left (a) Fixed programmed con-<br>trol, (b) Fuzzy logic control. . . . .                                                         | 48 |

|      |                                                                                                                           |    |
|------|---------------------------------------------------------------------------------------------------------------------------|----|
| B.9  | Rear legs angle walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control. . . . .            | 48 |
| B.10 | Body position walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control. . . . .              | 49 |
| B.11 | Body angle walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control. . . . .                 | 49 |
| B.12 | Body angular acceleration walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control. . . . .  | 50 |
| B.13 | Front legs torque walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control. . . . .          | 50 |
| B.14 | Rear legs torque walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control. . . . .           | 51 |
| B.15 | Front legs angle walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . .          | 52 |
| B.16 | Rear legs angle walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . .           | 52 |
| B.17 | Body position walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . .             | 53 |
| B.18 | Body angle walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . .                | 53 |
| B.19 | Body angular acceleration walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . . | 54 |

|                                                                                                                        |    |
|------------------------------------------------------------------------------------------------------------------------|----|
| B.20 Front legs torque walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . . | 54 |
| B.21 Rear legs torque walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control. . . . .  | 55 |

# Abbreviations

**FSM** Finite State Machine [20–22](#)

**N** Negative [23](#)

**NL** Negative Large [23](#)

**NM** Negative Medium [23](#)

**P** Positive [23](#)

**PID** Proportional Integral Derivative [38](#)

**PL** Positive Large [23](#)

**PM** Positive Medium [23](#)

**ZE** Zero [23](#)

# Chapter 1

## Introduction

A major goal in the robotic industry is to mimic living organisms movements in order to balance a robot when travelling on uneven surfaces. Robots that are able to adapt with changing environments can be used in a variety of scenarios. Some scenarios include replacing high risk human jobs with robots which can save lives. Other scenarios include situations where robots perform tasks better than humans like a search and rescue robot that can operate through the night. Whatever the task that the robot is designed to accomplish, large amounts of design and development considerations need to be taken into account in order for a robot to be able to adapt to the environment variations present in the real world. Designing for such scenarios can be a tough problem to solve due to unknown variables which the robot will encounter throughout its deployment. With careful design and implementation, a reliable robot can be created that is able to adapt to its surroundings and perform a desired task. Simplification of the problem allows for separate systems to be developed to solve the different challenges that the robot will face. Navigation of uneven surfaces is one of these systems and is the focus of this project.

This project aims to design and simulate a rigid bodied 4-leg robot with 6 degrees of freedom that is able to dynamically adjust its foot positioning while walking on an uneven surface. The dynamic stability mechanism will allow the robot to do its best to maintain a level body position which will help keep its centre of mass from deviating too far from its level ground position. The problem will be simplified by making the assumption that only one foot of the robot will be on an object at any point in time with the rest of the feet on level ground. In addition to this the robot will only be able to perform forward movement with the proposed control system. Contact sensors will be placed on each of the robots feet. This will allow the system to sense when the robot touches the ground and take necessary actions to maintain stability. The adaptive control system will utilise fuzzy logic. It is important to note that the robot will perform a walking gait which means that at least two feet are touching the ground at any moment in time. Other gaits including trotting and running would require changes to the movement algorithm for the robot to maintain balance.

Fuzzy logic is widely used for control systems and is a great solution for when inputs are bounded and can be divided into various membership functions. These systems produce valid and expected outputs given a specific uncertain bounded input. Integrating fuzzy logic into the control system of the robot allows for it to dynamically adjust its next movements based on the surface topology that it is stepping on. This allows for a more stable robot which can make more smooth forward progress compared to a fixed programmed robot. A block diagram of a simplified fuzzy logic system can be seen in [Figure 1.1](#).

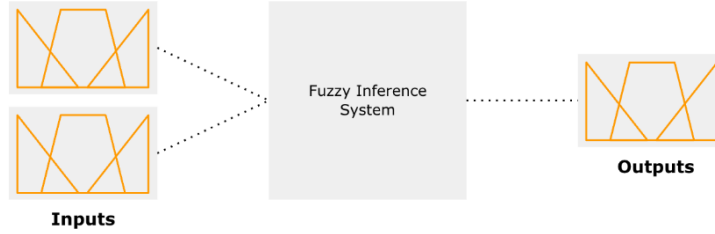


Figure 1.1: Fuzzy logic.

## 1.1 Problem Definition

Approaches to this problem include robots with 2, 4 and 6 legs. The developed robot will utilise a walking gait to make forward progress. In general, a walking robot needs to balance its center of gravity over a region of support which connects their legs that are touching the ground and can be seen in Figure 1.2. Due to its physical properties, a moving 6-leg robot with equally spaced legs can have three legs on the ground at any moment, forming a tripod with the center of mass located in the region of support. Unfortunately, the balancing of a robot with two and four legs in motion is more difficult because they don't have the ability to keep their center of mass inside the region of support without an adaptive stability system or wide foot pads.

When walking on a level surface the robot does not need to worry about its foot placement or how to make its next move. For this scenario a fixed programmed robot is able to make forward progress without tipping over. When the surface is not level, the robot will have to place its feet on top of objects which will affect its ability to maintain stability while making forward progress. A fixed programmed robot will struggle in this scenario and an adaptive algorithm is needed to keep the robot level while travelling over objects. The adaptive algorithm proposed in this project will use fuzzy logic. Fuzzy logic

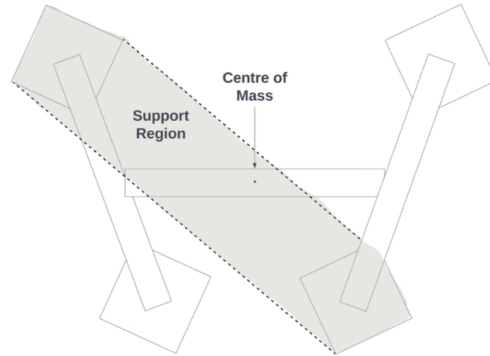


Figure 1.2: Robot centre of mass in support region.

allows for an intuitive rule based approach to solving the problem by representing inputs and outputs with various membership functions.

## 1.2 Objectives

The objectives of this project includes:

1. Research into 4 and 6-legged robot design
2. Design and implement a 3D model of the 4-legged robot that is stable while performing a walking gait
3. Implement a kinematic model which simulates interactions between the robot and the surface
4. Develop a method of controlling the robots movements (with and without the fuzzy logic controller)
5. Run simulations under various test conditions

6. Compare simulation results of the robot with fixed-programmed vs. fuzzy logic controlled movements and determine which control method is more desirable
  - Compare stability
  - Compare motor fatigue

# Chapter 2

## Literature Survey

A literature survey was conducted to research the different ways that this project could be designed and implemented using knowledge gathered from various sources. The project started by understanding and implementing a fuzzy logic controlled vehicle backup system similar to [2]. In addition to this a 6-leg fuzzy logic controlled robot outlined in [1] was developed. Implementing the backup system and the 6-leg robot allowed for a better understanding of the problem and proposed solution. Using findings from these projects, a 4-leg robot was able to be created.

An important component in this projects design is the determination of a walking gait that will allow it to maintain stability. [3] presents spring-mass modelling approaches for different animal gaits. [4] and [5] present 4-legged robots with statically stable gaits when travelling on a level surface. They do this through careful robot design and strategic foot placement. Since the author of this project had not used Simulink Multibody before, [6] was studied as it presented a project which created a 4-legged robot using Simulink Multibody. When designing a robot with articulating joints, it is important to be able to determine

the absolute position of those joints with a high degree of accuracy. [7] presents a method of improving the accuracy of determining positions, velocities and accelerations of joints in robotic arms. Controlling a robots joint actuation can become complicated when there are multiple joints that need to be actuated to achieve the desired motion. To achieve the desired position, kinematic equations can model this motion and provide values for joint angles. [8] and [9] present quadruped joint designs using inverse kinematics equations. The final domain which was surveyed was designing fuzzy logic systems in order to create the control system for the robot. [10] implements a 6-legged robot navigation algorithm using fuzzy logic. Further research on terrain identification and prediction using fuzzy logic was done with insights gained from [11] and [12]. Completing this literature survey allowed for a better understanding of the problem in order to be able to develop the solution outlined in this report.

# Chapter 3

## Proposed Solution

### 3.1 Robot Design

The solution that was designed for this project has four equally spaced legs, each with foot pads, to help the robot maintain stability. The robots four legs are attached to create one leg section for the front legs and one for the back legs. This was done to simplify the control system needed to move the robot. It is important to note that all the components of the robot are rigid. The robot was created with the majority of its mass located in its body with a negligible amount of mass in its legs. This in combination with the foot pads allow for the center of mass to be located inside the support region when the robot has two feet on the ground as seen in Figure 1.2. This allows the robot to maintain stability and not tip over.

Each leg has two degrees of freedom represented by the angles Alpha ( $\alpha$ ) and Beta ( $\beta$ ). The angle Alpha ( $\alpha$ ) represents the angle that a specific leg section is rotated up or down as seen in Figure 3.1. The robot has 2 leg sections so there are two Alpha ( $\alpha$ ) angles,

Alpha1 ( $\alpha_1$ ) and Alpha2 ( $\alpha_2$ ), representing the front and rear legs rotation. The angle Beta ( $\beta$ ) represents the lateral rotation of the robots legs in relation to the body as seen in Figure 3.2. Similar to the angle Alpha ( $\alpha$ ), Beta ( $\beta$ ) has two angles which represent the angles of the front and rear leg sections. These angles are Beta1 ( $\beta_1$ ) and Beta2 ( $\beta_2$ ). Given the movement algorithm which will be described in Section 3.2, the angle Beta1 ( $\beta_1$ ) will always be equal to negative Beta2 ( $\beta_2$ ) and vice versa. It will be seen that we will only keep track of Beta ( $\beta$ ) for the front leg section and just negate Beta ( $\beta$ ) before feeding into the rear legs for simplicity.

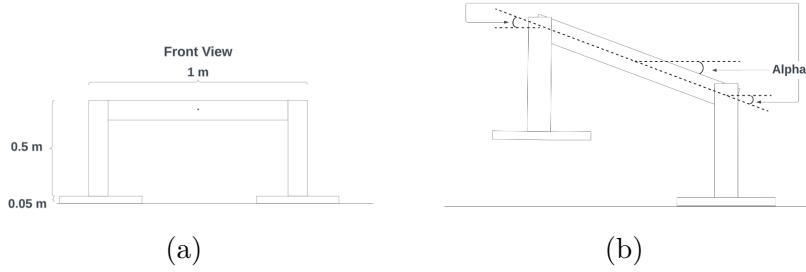


Figure 3.1: Robot front view (a) On level ground, (b) In motion.

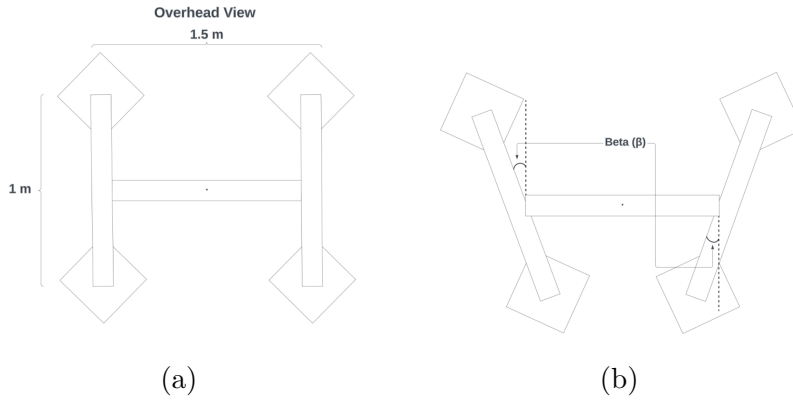


Figure 3.2: Robot overhead view (a) On level ground, (b) In motion.

## 3.2 Robot Movement

The robot will follow a six step walking movement sequence seen in Figure 3.3. This sequence ensures that at least two feet are on the ground at any moment which allows the robot to remain stable while in motion. The robot starts in its initial state with the front legs at angle Beta ( $\beta$ ) and the rear legs at angle negative Beta ( $\beta$ ). Then the robot lifts up its front right and rear left legs by changing each legs angle of Alpha ( $\alpha$ ). In the next step, the robot changes each legs angle of Beta ( $\beta$ ) to move the two legs that are on the ground backwards. This is 1 of 2 states in the movement design that makes the robot move forward. These states are seen in green. All of the legs are then moved to the ground and the sequence continues by moving the front left and rear right legs up off the ground. Then the front right and rear left legs are moved backwards. All of the legs are then moved to the ground and the movement sequence is complete. The sequence is repeated as long as the simulation is running to achieve forward motion.

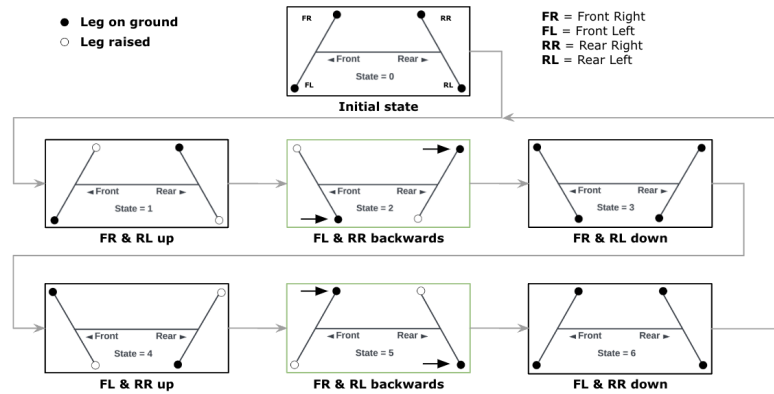


Figure 3.3: Movement sequence.

### 3.3 Simulation Overview

The robot was simulated using MATLAB's Simulink Multibody tool. This tool includes various components which allow for 3D design and simulation of mechanical systems. Simulating the project with a virtual tool allows for quick iteration and debugging as system performance can be evaluated with a variety of sensors. These sensors can be placed anywhere in the design and can give insights on how the system is performing compared to expectations. The final design of the robot and environment can be seen in Figure 3.4. It includes various subsystems connected together to implement a modular design methodology. This allows for easier debugging as well as the reuse of similar components in the project. The robot has two sets of legs and one body. The ground plane is created with a Grid Surface component and is connected to both sets of legs to allow for the system to sense contact between the ground and the legs. The design contains two separate controllers to control the robots movement and stop the legs once one set contacts the ground. In addition to this there is two scope components which allows for data logging via the various traces that come from the components of the robot. To analyse the performance of the proposed fuzzy logic control system verses fixed programmed movements, various parameters of the robot will need to be measured. These parameters include position, angle, angular velocity and angular acceleration of the body in relation to the ground plane. In addition to this, the torque required for motor movement will be measured. This torque represents the amount of force required to maintain the desired angles of Alpha ( $\alpha$ ) and Beta ( $\beta$ ). This also gives insight into the power consumption of the robot in the various simulations due to the amount of motor effort that is being put into maintaining positioning. The simulations performed were each 100 seconds in length to ensure that the robot had enough time to reach and traverse over the object.

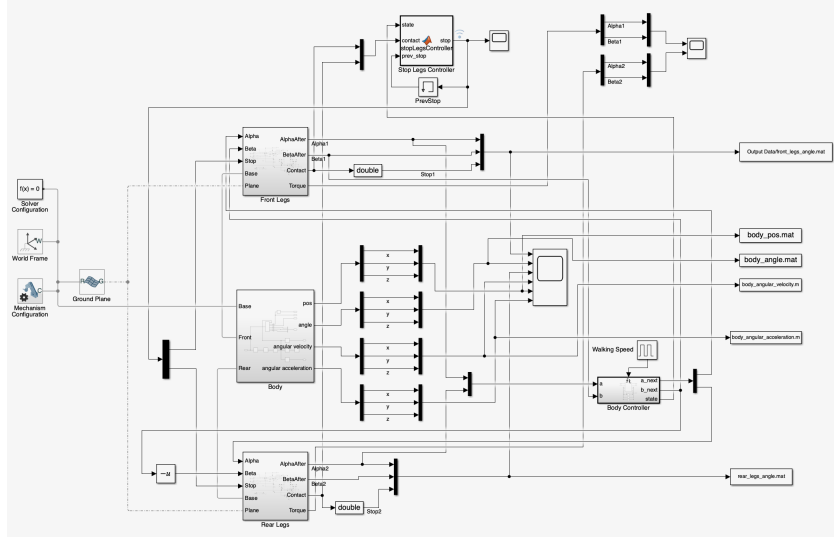


Figure 3.4: Simulation overview.

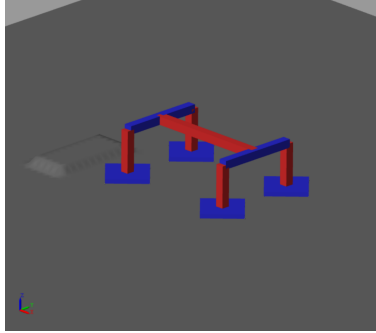


Figure 3.5: 3D simulation.

### 3.4 Ground Initialisation

Initialising the ground plane is necessary in order for the robot to have different surface features to walk over. The choice between surface features and the locations of these features is controlled by the ground initialisation code seen in Figure A.1. This code sends an  $N \times M$  matrix to the Grid Surface component where  $N$  and  $M$  are defined by (3.1) and

(3.2) where the width and length represent the size of the ground plane in metres. For this project the resolution was set to 0.1 m to improve performance by limiting ground plane calculations and to be able to run the simulations on a generic PC without crashing. The matrix contains the x and y coordinates of each point as well as the height z at that point.

$$N = 2\left(\frac{width}{resolution}\right) + 1 \quad (3.1)$$

$$M = 2\left(\frac{length}{resolution}\right) + 1 \quad (3.2)$$

To create a level surface with no obstacles, all values in the *init\_ground* array are set to 0. By specifying the x, y and z components at different points on the plane, an obstacle can be created for the robot to walk over. The obstacles were chosen to have a height of 0.1 metres through experiments to determine the robots capabilities in successfully traversing objects with minimal instability. The position of the object can then be specified by changing the variable *obstacle\_pos* to either "L" or "R". This specifies whether the obstacle will be on the left or right side of the robot. When the obstacle is on either side of the robot only one leg will come in contact with the obstacle at any moment during the simulation. After running the ground initialisation code, a global variable will be saved to the MATLAB workspace in a variable named *ground\_z* which will be read by the grid surface component in Simulink to initialise the ground plane.

### 3.5 Body Subsystem

The body subsystem represents the main body of the robot and contains all of the robots mass. This is done to improve its stability by not having large moving masses far from the

center of gravity of the robot. The body's dimensions, density and weight can be seen in Table 3.1. The density of the body was left at its default value of 5000 kg/m<sup>3</sup>. This gives a calculated weight of 75 kg given the body's dimensions.

| Body Dimensions<br>(m) |     |     | Density (kg/m <sup>3</sup> ) | Weight (kg) |
|------------------------|-----|-----|------------------------------|-------------|
| x                      | y   | z   |                              |             |
| 1.5                    | 0.1 | 0.1 | 5000                         | 75          |

Table 3.1: Body properties.

Figure 3.6 shows the body subsystem created in MATLAB Simulink. The system has one input and six outputs. The robot has six degrees of freedom, meaning that it has the same rotational and translational freedom that exists in the real world. That is what the six degree of freedom joint in the body subsystem creates. There are a number of translational rigid transforms in this subsystem which allows the system to output coordinate reference frames for the front and rear sets of legs. There is also a rigid transform that makes the robot start from a height of 0.5 m in the positive z direction, which is above the ground plane, at the start of the simulation. This allows the robot to fall slightly at the start of the simulation to ensure that the body starts on the positive z side of the ground plane. The last four sensor outputs get their values from a transform sensor. This sensor measures the robots movement relative to the world frame which is given by the Base input. Each of these outputs contain three values for their respective x, y and z components. In order to obtain the angle measurements from the robot, the angular velocity measurement is sent through an integration block before being sent as an output. The body geometry can be seen in Figure 3.7.

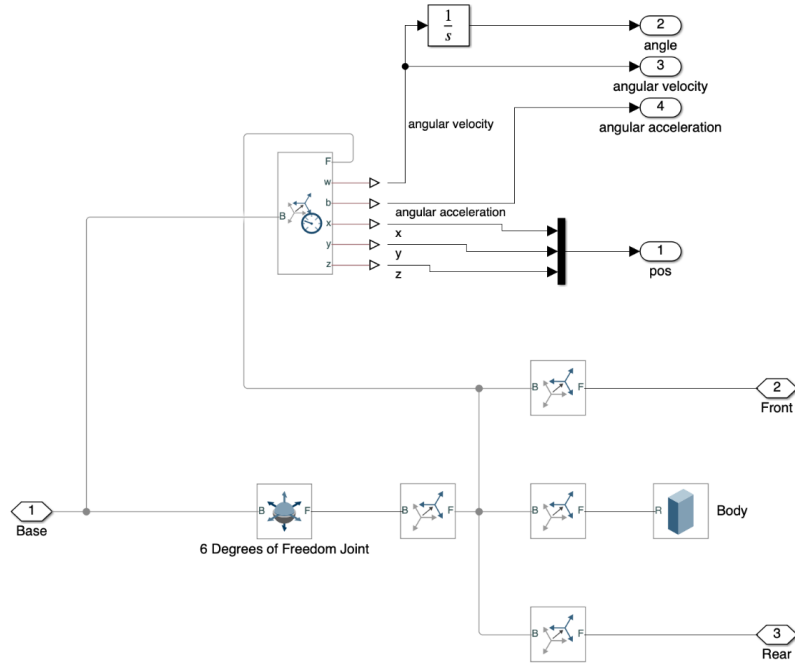


Figure 3.6: Body subsystem.

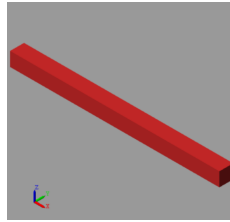


Figure 3.7: Body geometry.

### 3.6 Front and Rear Legs Subsystem

The front legs subsystem represents the legs of the robot and includes the lateral support arm, two vertical legs and two foot pads with contact sensors. This subsystem can be seen in Figure 3.8. The front and rear leg subsystems are identical. All of the items in this subsystem have a negligible amount of mass which improves the robots stability.

This subsystem is the most complex because it is what defines the kinematic model of the robot as the legs are the components being actuated to achieve forward movement. The subsystem has five inputs and four outputs. The five inputs include the desired angles Alpha ( $\alpha$ ) and Beta ( $\beta$ ) as well as a stop signal which stops each set of legs from moving once a leg section has both feet on the ground. The base input represents the coordinates given by the body subsystem which is either at the front or rear of the robot. The final input is the plane which represents the ground that the robot is resting on. This allows for the contact sensors to detect contact between the feet and the ground and take action. The four outputs include the actual angles AlphaAfter ( $\alpha'$ ) and BetaAfter ( $\beta'$ ) as well as a contact signal. The subsystem also outputs torque sensing data for the joints representing Alpha ( $\alpha$ ) and Beta ( $\beta$ ). This output represents the torque required to reach or maintain the motor angle supplied to the subsystem. The geometry of the leg subsystem can be seen in Figure 3.9.

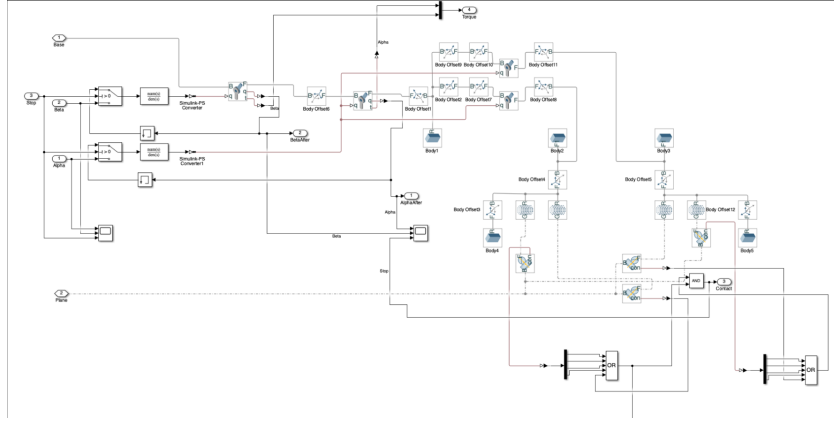


Figure 3.8: Front or rear legs subsystem.

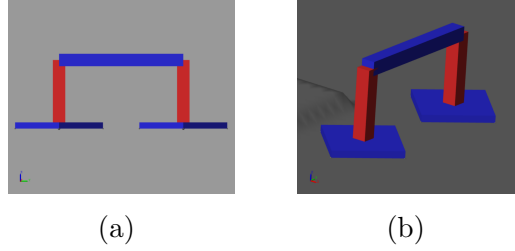


Figure 3.9: Leg geometry (a) Front view, (b) Perspective view.

### 3.6.1 Leg Actuation

A section of the leg subsystem that focuses on the actuation of the Alpha ( $\alpha$ ) and Beta ( $\beta$ ) motors can be seen in Figure 3.10. Actuation of the motors representing Alpha ( $\alpha$ ) and Beta ( $\beta$ ) is achieved by receiving a desired input angle from the subsystem. Upon receiving this angle, the system only actuates the motor if the stop input is not equal to one. This is done through a relay which controls whether the motor receives the new angle or the previous angle that the motor was at. If the motor receives the previous angle that it was at then it will no move. This is done for both motors for Alpha ( $\alpha$ ) and Beta ( $\beta$ ) and is essential in order to stop motor actuation once the system senses that both feet are touching the ground. Before sending the desired angle for Alpha ( $\alpha$ ) or Beta ( $\beta$ ) into the motors input, the angle goes through a first order transfer function defined by (3.3). The purpose of this transfer function is to smooth abrupt changes in angle so that the robot does not lose its balance. The transfer function was chosen experimentally to ensure that the output reaches its final value within two seconds. This is important because the body controller system makes movements every two seconds which means the legs need to reach their desired positions before the next move. The angle input can be represented by a step function where its final value is the set angle by the body controller. Figure 3.11 shows the step input and the output after the transfer function smooths the abrupt change in angle.

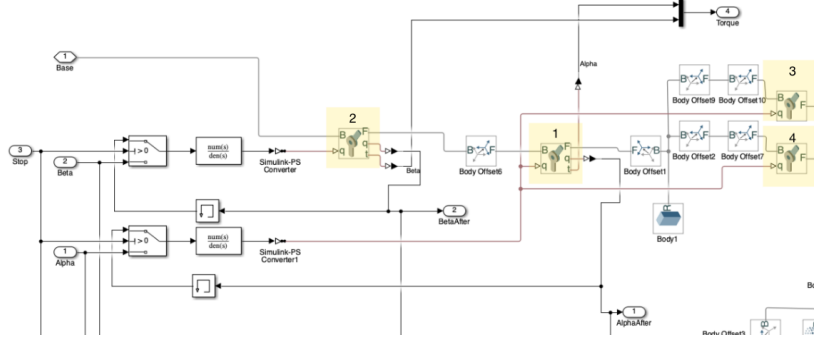


Figure 3.10: Front or rear legs subsystem actuation.

$$H(s) = \frac{1}{0.2s + 1} \quad (3.3)$$

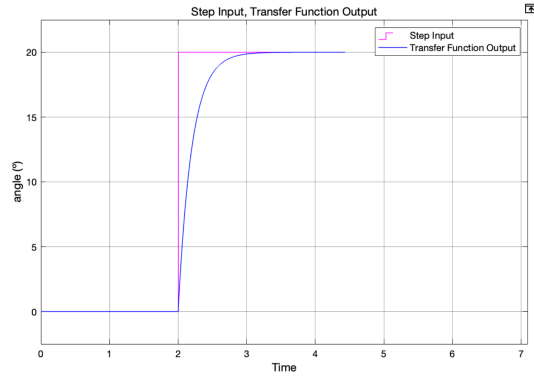


Figure 3.11: Step input and smoothing transfer function output.

The revolute joints in this subsystem rotate around their z-axis so a series of rotational transformations are done before each block to ensure that their z-axis is lined up correctly to achieve the desired actuation. Looking at Figure 3.10 and Figure 3.12, it can be seen where each revolute joint is on the robots body. In order for the foot pads to stay level while the robot raises its lateral leg supports, rotation of the two vertical legs is needed. These motors are represented by the numbered revolute joints 3 and 4 in Figure 3.10 and

Figure 3.12. Analytically, it can be seen from Figure 3.1 that these joints should maintain an angle of Alpha ( $\alpha$ ) in order to keep them vertical throughout the robots movements. Each of these joints actuate based on their applied input angle, with the force required to reach the desired input angle automatically computed by MATLAB during the simulation. In order to determine the actual angle that the joint is currently at, the Alpha ( $\alpha$ ) or Beta ( $\beta$ ) joints have sensing outputs which are sent as outputs of the subsystem to the body controller. The sensing outputs are *AlphaAfter* and *BetaAfter*. All of the angles are converted from radians to degrees through the property editor of each joint block for a more readable debugging experience.

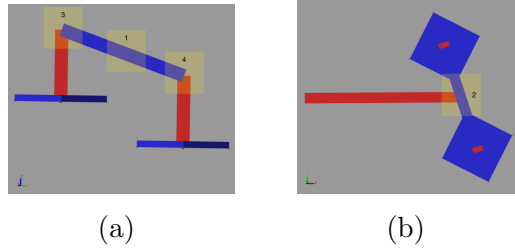


Figure 3.12: Leg actuation (a) Side view, (b) Top view.

### 3.6.2 Contact Sensing

The leg subsystem also contains contact sensors which sense contact between the leg section and the ground. Each leg section will have its own identical contact sensing system which act independently from each other. Each foot pad has five contact sensors which can be seen in Figure 3.13 and is said to be in contact with the ground if any of them are touching the surface. This improves the stability of the robot when it is not entirely on the object it is stepping on by stopping the leg upon contact. In this scenario the foot pad is partially supported by the object with some of the foot pad not touching the ground. The contact

signal is set to one if both sides of a leg section are in contact with the ground. This signal is then used by the stop legs controller outlined in Section 3.7 to stop the legs from actuating too far when travelling over an object.

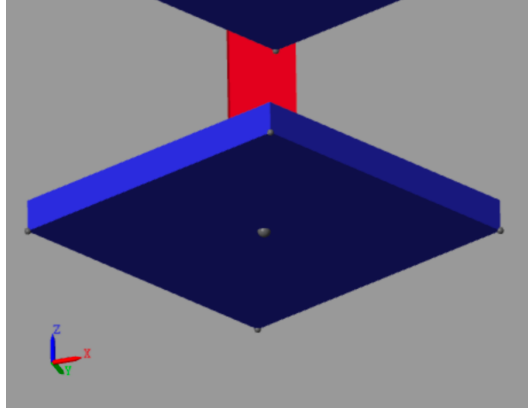


Figure 3.13: Foot contact sensors.

### 3.7 Stop Legs Controller

The stop legs controller implements a [Finite State Machine \(FSM\)](#) where its state is determined by the body controller subsystem outlined in Section 3.8. This subsystem can be seen in Figure 3.14 and the MATLAB code implementation can be seen in Figure A.2. The purpose of this system is to stop leg movement once both sides of the front or rear legs make contact with the ground. This is needed because an objective of this project is to try to maintain a level body position when walking over objects. The system has three inputs and one output. The *state* input receives the current state of the body controller. The controller is only active during the two states where the legs are instructed to go down by the body controller which are the third and sixth state. The *contact* input receives a two row array representing the front and rear contact signals which are set to one by the leg

subsystem if both feet in that system are in contact with the ground. The *previous\_stop* input also receives a two row array except it is used to determine if a leg has already been stopped while in the same state. This is needed because the robot has six degrees of freedom which causes the contact signal from the leg subsystem to oscillate as the legs are not perfectly level. In short, once a contact signal is received, the system ignores the signal until the next state change. In this case, the system assumes that the legs are still in contact with the ground. The *stop* output also receives a two row array and sets each row to one or zero which triggers the relays in the leg subsystem to stop moving the legs and maintain their current position.

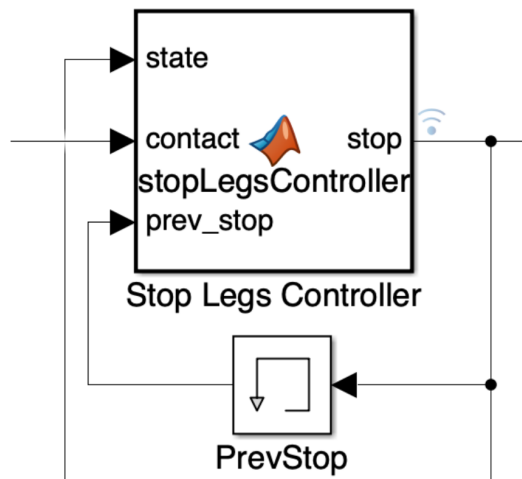


Figure 3.14: Stop legs controller subsystem.

## 3.8 Body Controller

The body controller subsystem implements the main movement controller [FSM](#) for the robot and can be seen in [Figure 3.15](#). The MATLAB code implementation of this system

can be seen in Figure A.3. Joint actuation can be achieved either through fixed programmed movements or by utilising the fuzzy logic system that will be described in Section 3.8.1. Controlling this choice is achieved through modification of the *ENABLE\_FIS* variable. Setting this variable to one enables the fuzzy logic control system. The FSM goes through the six stage movement design outlined in Section 3.2. State changes are performed every two seconds which is controlled by a pulse generator with a period of 4 seconds and a 50% pulse width. The pulse generator enables the trigger in the body controller subsystem seen in Figure 3.15 to change states. The system is triggered on the rising and falling edge of the pulse meaning the robot will make movement every two seconds. When the fuzzy controller is enabled, the system uses fuzzy logic when raising its legs to maintain better balance when walking over surface features. It sends the current angles of the joints and gets the next angles that it needs to actuate Alpha ( $\alpha$ ) and Beta ( $\beta$ ) to stay as level as it can and minimise motor stress. The system has five inputs and five outputs. It receives the current angles Alpha ( $\alpha$ ) and Beta ( $\beta$ ) as well as the last angle of Alpha ( $\alpha$ ) that the front and rear legs stopped at. This information is used by the controller to know how far to actuate Alpha ( $\alpha$ ) on the next downward movement to be able to have both legs in a section touching the ground. It also receives the next state in order to move through the FSM. The system outputs the next values for Alpha ( $\alpha$ ) and Beta ( $\beta$ ) as well as the last angle that the front and rear legs stopped at. It also outputs the next state which is sent back into the system.

### 3.8.1 Fuzzy Logic System

The fuzzy logic system used to control the robot implements a Mamdani Type-1 fuzzy logic controller and is adapted from [1] to fit this projects design. The system has three

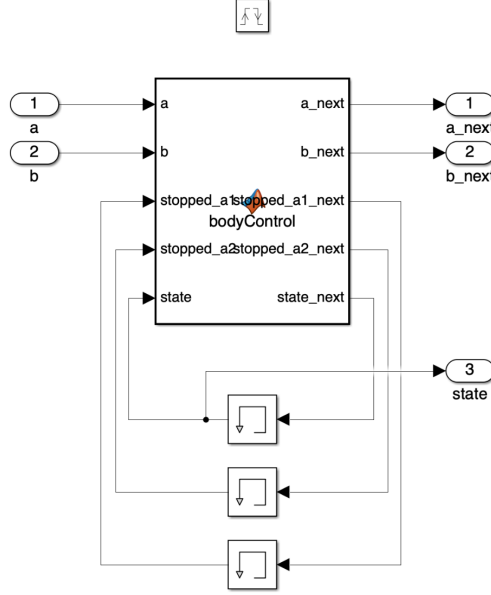
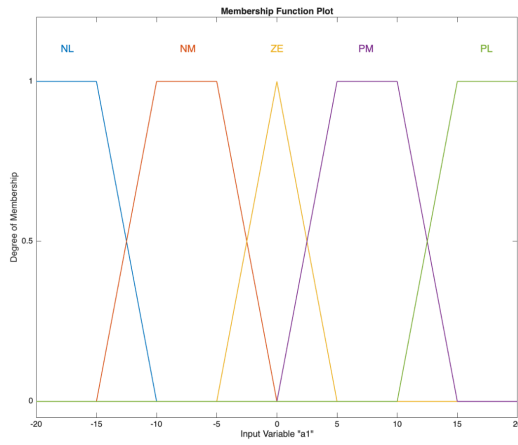


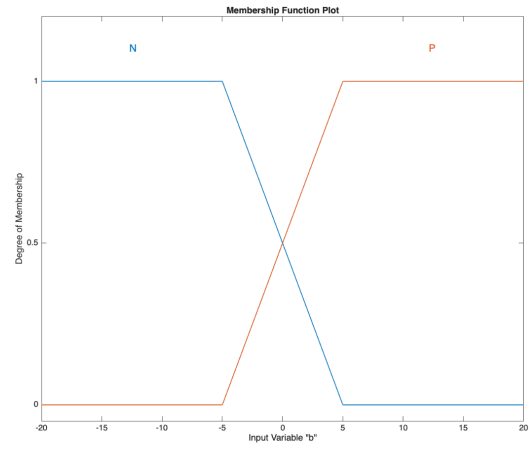
Figure 3.15: Body controller subsystem.

inputs and two outputs. The first two inputs are Alpha1 ( $\alpha_1$ ) and Alpha2 ( $\alpha_2$ ). They both have a range of  $-20^\circ$  to  $20^\circ$  and are broken down into five membership functions. These are **Negative Large (NL)**, **Negative Medium (NM)**, **Zero (ZE)**, **Positive Medium (PM)** and **Positive Large (PL)**. The membership function shapes were chosen to be trapezoidal for simplicity and can be seen in Figure 3.16a. The third input is Beta ( $\beta$ ) which has the same range of  $-20^\circ$  to  $20^\circ$  and is defined by the two membership functions **Negative (N)** and **Positive (P)**. The membership functions shapes of this input were also chosen to be trapezoidal and can be seen in Figure 3.16b. The two outputs are Alpha1' ( $\alpha_1'$ ) and Alpha2' ( $\alpha_2'$ ) with the same range. These outputs are broken down into the same 5 membership functions but as singleton values and can be seen in Figure 3.16c. The fuzzy rule set can be seen in Tables 3.2 to 3.4 and is designed to keep the robot more stable than fixed-programmed movements when travelling over an object. It is important to note

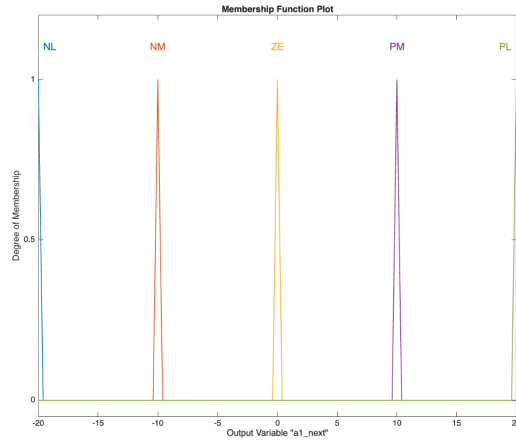
that the fuzzy logic system was designed for only one of the legs to be stepping on an object at any point in time with the other legs on level ground. This simplifies the fuzzy rules needed to simulate and test the project. In order understand how the fuzzy logic system will respond to various input conditions, it is useful to create a control surface which goes through various input values and plots output values on a three-dimensional graph. Doing so allows for a visual representation of the fuzzy logic controller which allows for the designer to find potential problems with the design by finding and correcting the rules that are creating the undesirable output value. The notable control surfaces relevant to the rule base presented for this project can be seen in [Figure 3.17](#).



(a)



(b)



(c)

Figure 3.16: Membership functions (a) Input angle Alpha1 or Alpha2 ( $\alpha_1$  or  $\alpha_2$ ), (b) Input angle Beta ( $\beta$ ), (c) Output angle Alpha1' or Alpha2' ( $\alpha_1'$  or  $\alpha_2'$ ).

Table 3.2: Fuzzy rules with robot on level surface [1]

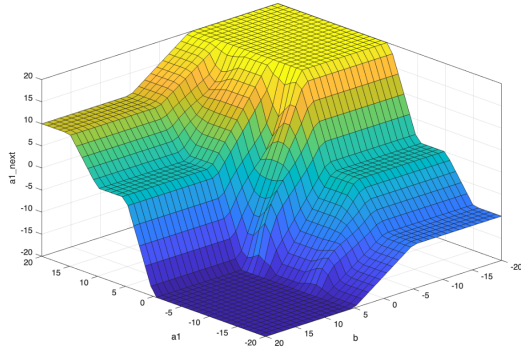
|                   |    | Beta  |       |
|-------------------|----|-------|-------|
|                   |    | N     | P     |
| Alpha 1 or Alpha2 | ZE | PL/NL | NL/PL |

Table 3.3: Fuzzy rules with front leg on object [1]

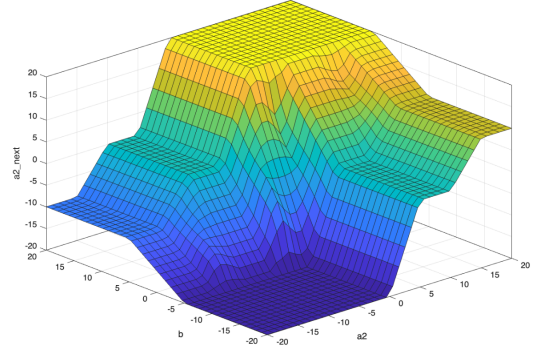
|               |           | <b>Beta</b> |          |
|---------------|-----------|-------------|----------|
|               |           | <b>N</b>    | <b>P</b> |
| <b>Alpha1</b> | <b>NL</b> | NM/NL       | NL/PL    |
|               | <b>NM</b> | ZE/NM       | NL/PL    |
|               | <b>PM</b> | PL/NL       | ZE/PM    |
|               | <b>PL</b> | PL/NL       | PM/PL    |

Table 3.4: Fuzzy rules with rear leg on object [1]

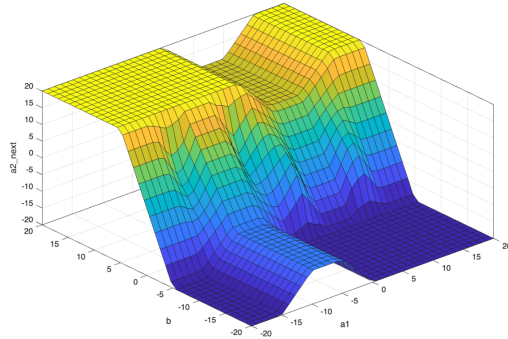
|               |           | <b>Beta</b> |          |
|---------------|-----------|-------------|----------|
|               |           | <b>N</b>    | <b>P</b> |
| <b>Alpha2</b> | <b>NL</b> | PL/NL       | NL/NM    |
|               | <b>NM</b> | PL/NL       | NM/ZE    |
|               | <b>PM</b> | PM/ZE       | NL/PL    |
|               | <b>PL</b> | PL/PM       | NL/PL    |



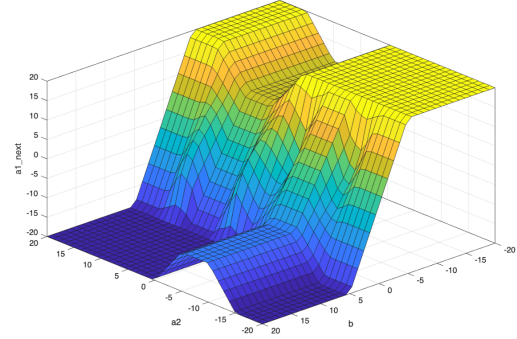
(a)



(b)



(c)



(d)

Figure 3.17: Control surfaces (a) Alpha1 vs. Beta vs. Alpha1' ( $\alpha_1$  vs.  $\beta$  vs.  $\alpha_1'$ ), (b) Alpha2 vs. Beta vs. Alpha2' ( $\alpha_2$  vs.  $\beta$  vs.  $\alpha_2'$ ), (c) Alpha1 vs. Beta vs. Alpha2' ( $\alpha_1$  vs.  $\beta$  vs.  $\alpha_2'$ ), (d) Alpha2 vs. Beta vs. Alpha1' ( $\alpha_2$  vs.  $\beta$  vs.  $\alpha_1'$ ).

# Chapter 4

## Results

Each simulation was run for 100 seconds which allowed for enough data to be collected for a conclusion to be made about the designed system. In Sections 4.1 to 4.3, simulation results of various measured parameters will be presented with snapshots from simulation videos to show a clear representation of the results obtained. The results data for each simulation can be seen in Appendix B. Each environment setup was run twice to allow the robot to be simulated with fixed programmed as well as fuzzy logic controlled movements.

### 4.1 Level Surface

The first simulations that were run involve the robot walking on a level surface. Although not very interesting, it serves as a benchmark to ensure that the fuzzy logic controlled robot performs as expected when travelling on level ground. The fixed programmed movement robot had hard-coded movement angles based on the designed movement sequence. Figures B.1 and B.2 show very similar results in front and rear angles of Alpha ( $\alpha$ ) between

the fixed programmed and fuzzy logic controlled robot which is the desired result. Screenshots of the simulation running can be seen in [Figure 4.1](#) and [Figure 4.2](#) with video found externally at [Fixed Programmed Robot Walking on Level Surface](#) and [Fuzzy Controlled Robot Walking on Level Surface](#)

Comparing body position as seen in [Figure B.3](#), similar results can be seen between the two designs. Both designs are relatively stable and both make significant progress in the negative x direction. It is noted that in both designs there is a slight unintentional drift in the negative y direction of movement. This drift is caused by the mechanical design of the robot and is not the focus of this project. All that this means is that the robot veers slightly away from a perfectly straight path while making forward progress.

Comparing body angle and angular acceleration, as seen in [Figures B.4](#) and [B.5](#), allows for an analysis of the stability of the robot by measuring how it's affected by walking over surface features. The following comparisons show little difference between the two control methods when travelling on a level surface other than the drift in body angle which was commented on earlier.

The final comparison for each simulation will be the torque required to maintain motor position for the front and rear legs. This statistic allows for a quantitative analysis of the required effort and subsequent power consumption of the motors in each design. As seen in [Figures B.6](#) and [B.7](#), on level ground, the two designs have nearly identical torque profiles for all motors.

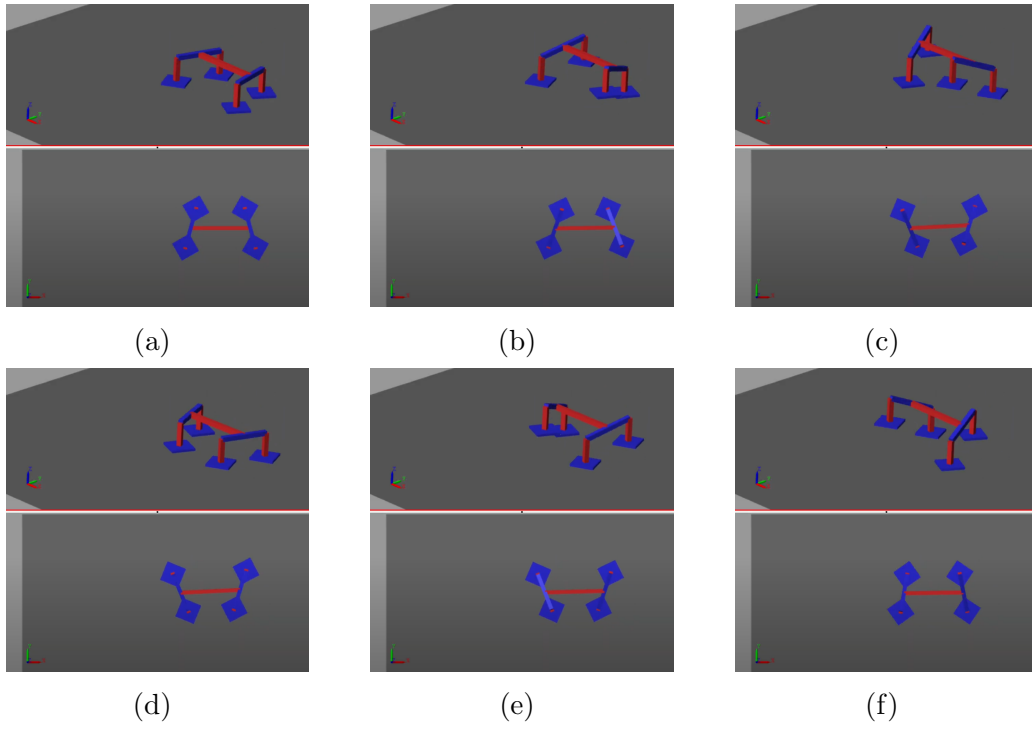


Figure 4.1: Fixed programmed simulation screenshots on level surface (a) time=4s, (b) time=6s, (c) time=8s, (d) time=10s, (e) time=12s, (f) time=14s.

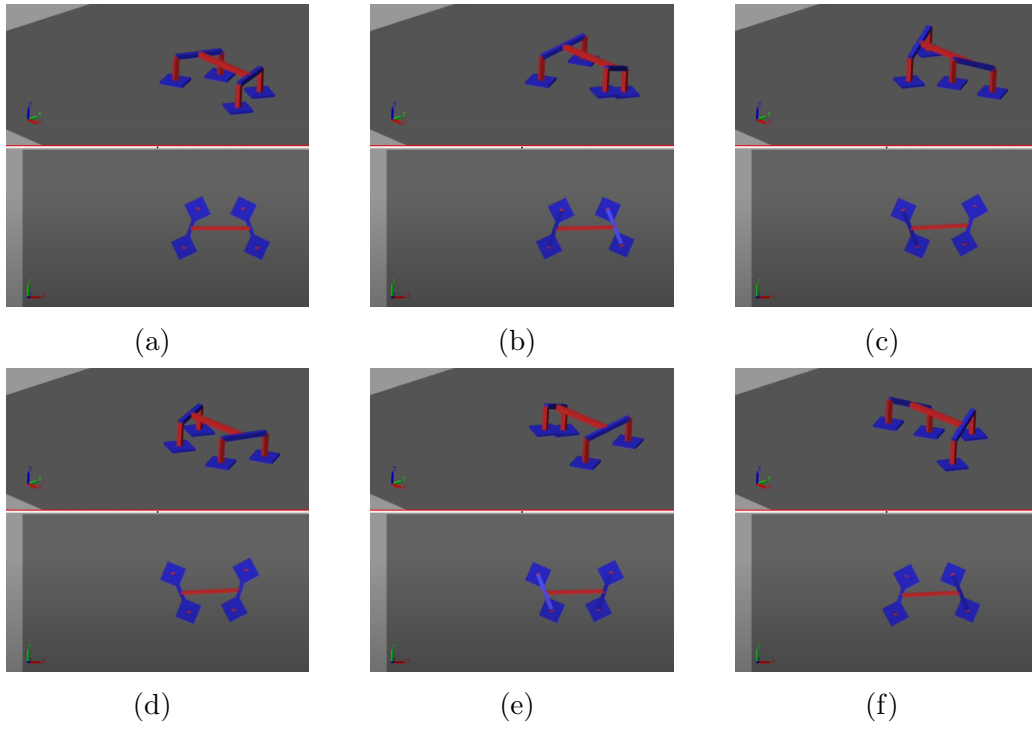


Figure 4.2: Fuzzy logic controlled simulation screenshots on level surface (a) time=4s, (b) time=6s, (c) time=8s, (d) time=10s, (e) time=12s, (f) time=14s.

## 4.2 Object Under Left Legs

The next simulations were run while placing an object on the ground to the left of the robot so it can walk over it. This is where the differences between the fixed programmed and fuzzy logic controlled robot can be seen. Screenshots of the simulation running can be seen in [Figure 4.3](#) and [Figure 4.4](#) with video found externally at [Fixed Programmed Robot Walking Over Object on Left](#) and [Fuzzy Controlled Robot Walking Over Object on Left](#). Comparing front and rear leg angles Alpha ( $\alpha$ ) in [Figures B.8](#) and [B.9](#), it can be seen that the fuzzy logic controlled movement robot adapts its next movement angle based on the obstacle it is stepping on. It is important to note that the fixed programmed movement robot still stops its leg movement while moving its legs to an Alpha ( $\alpha$ ) angle of zero. This shows the angle at which the object stopped the legs with both foot pads touching the ground. Although this helps the fixed programmed robot by maintaining contact with the ground for as long as possible, it does not alter its next movements and just actuates its motor to the default large value of Alpha ( $\alpha$ ). This causes the robot to be less stable due to the over extension of its legs for the surface it is walking on.

Comparing body position data as seen in [Figure B.10](#), it is clear that the fuzzy logic controlled robot significantly smooths x, y and z translational movements. Although both designs make significant forward progress in the negative x direction, the fixed programmed movement design suffers from instability when traveling over the object when compared to the fuzzy logic controlled robot.

Viewing the body angle and angular acceleration data, seen in [Figures B.11](#) and [B.12](#), of the two designs, it can be seen that the fuzzy logic controlled robot performs better. Other than the general drift previously mentioned, the body angle in the x and y directions is better controlled and kept to a minimum. Looking at the angular acceleration, the fixed

programmed movement robot has a much larger average value compared to the fuzzy logic controlled movement. In this context, a lower angular acceleration of the body is more desirable because we want it to try to maintain a near zero angular acceleration to maintain balance.

Analysis of the motor torque for each design seen in Figures B.13 and B.14 shows a clear improvement in the average torque required to maintain motor positioning in the robot. It can be seen how the fuzzy logic controlled robot is able to perform the same task of traversing over the object while using less power and putting less strain on its motors.

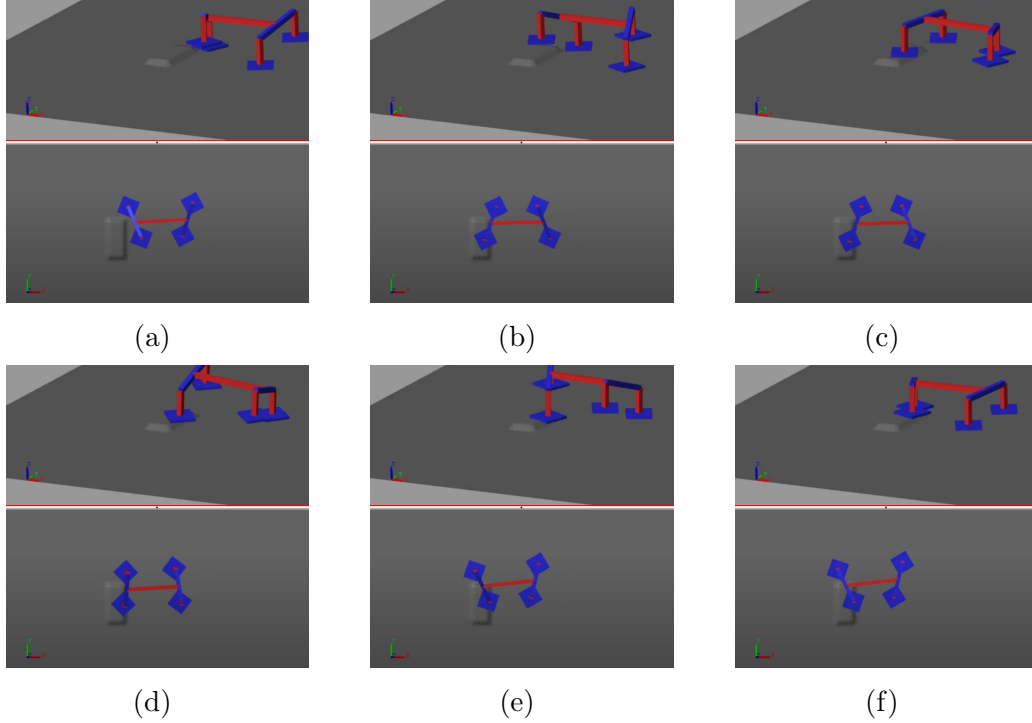


Figure 4.3: Fixed programmed simulation screenshots with object under left legs (a) time=12s, (b) time=14s, (c) time=16s, (d) time=18s, (e) time=20s, (f) time=22s.

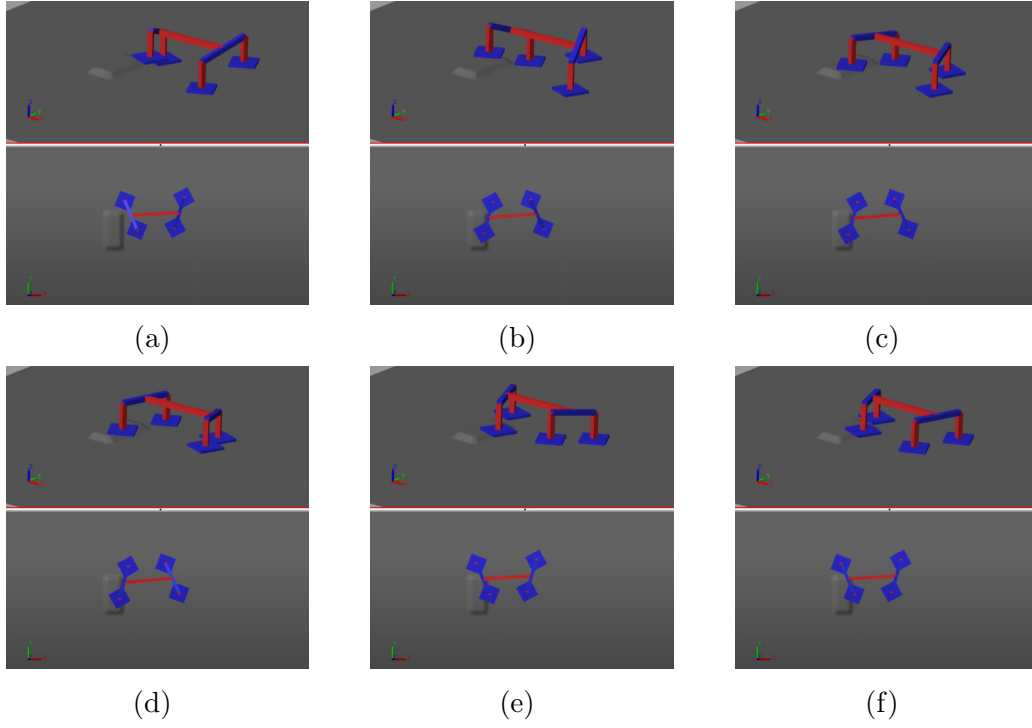


Figure 4.4: Fuzzy logic controlled simulation screenshots with object under left legs (a) time=12s, (b) time=14s, (c) time=16s, (d) time=18s, (e) time=20s, (f) time=22s.

### 4.3 Object Under Right Legs

Results in Figures [B.15](#) to [B.21](#) show similar results when placing the object on the right side of the robot for it to walk over. While traversing over the object, this data shows better control and balance with the fuzzy logic controlled movement versus the fixed programmed movement robot. In addition to this, motor fatigue and power usage is lower when the robot is controlled using the fuzzy logic system. Screenshots of the simulation running can be seen in Figure [4.5](#) and Figure [4.6](#) with video found externally at [Fixed Programmed Robot Walking Over Object on Right](#) and [Fuzzy Controlled Robot Walking Over Object](#)

on Right.

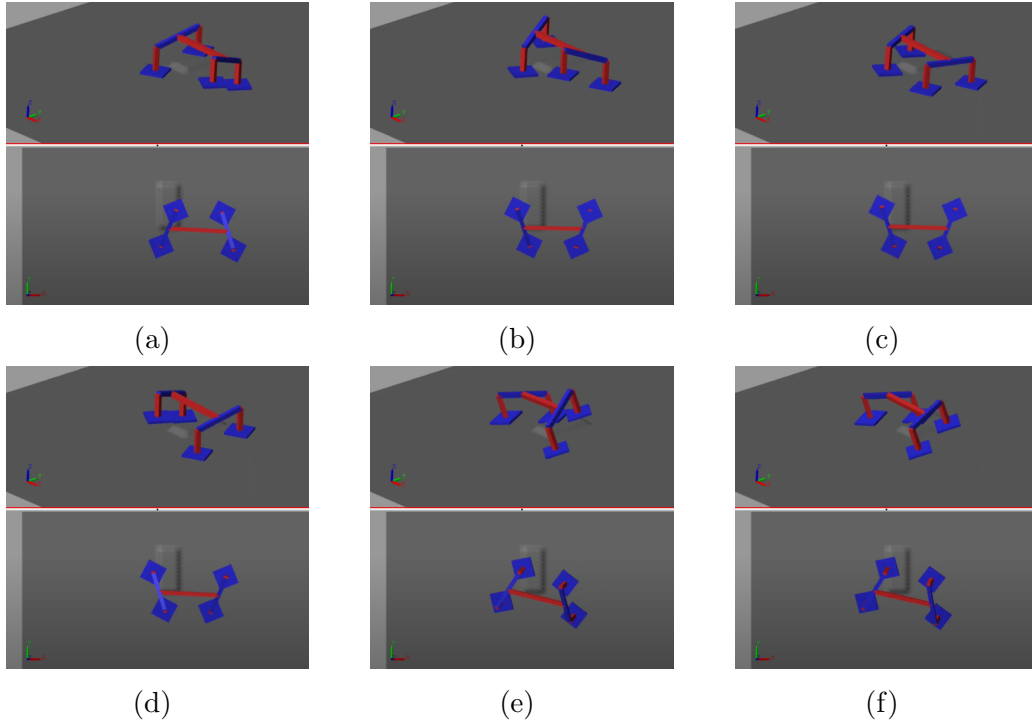


Figure 4.5: Fixed programmed simulation screenshots with object under right legs (a) time=18s, (b) time=20s, (c) time=22s, (d) time=24s, (e) time=26s, (f) time=28s.

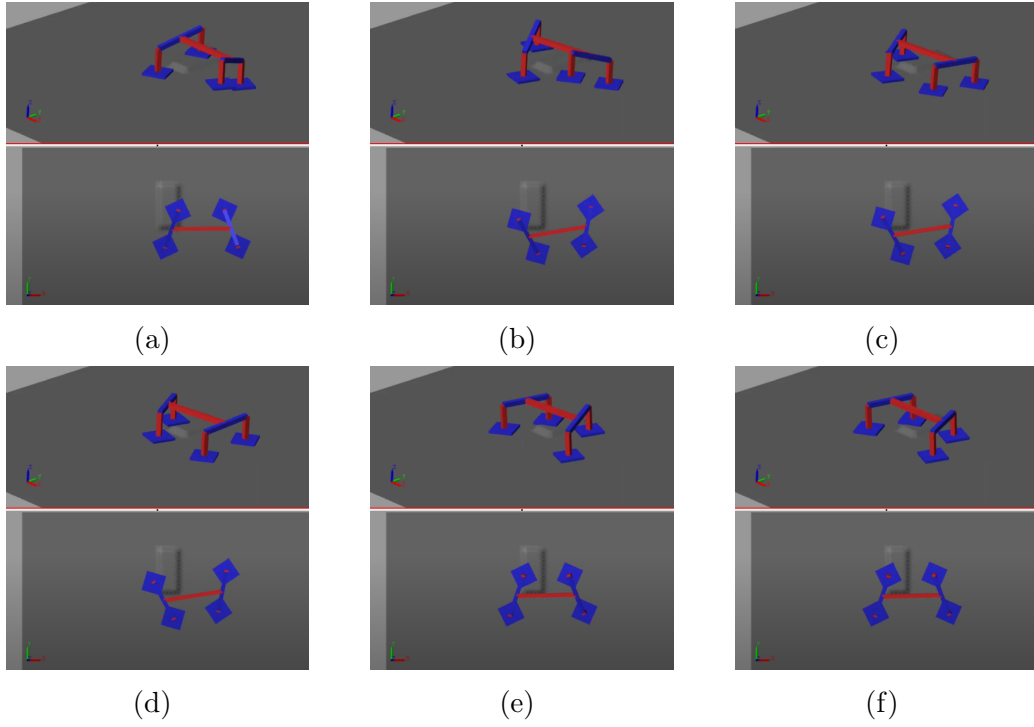


Figure 4.6: Fuzzy logic controlled simulation screenshots with object under right legs (a) time=18s, (b) time=20s, (c) time=22s, (d) time=24s, (e) time=26s, (f) time=28s.

# Chapter 5

## Future Work and Conclusion

### 5.1 Future Work

Due to the following limitations in the scope of this project, future work can be done to improve the design of the robot as well as its control system in order to obtain results that exceed the performance of this project. The fuzzy rule set was limited to only cover scenarios with on leg stepping on an object at a time. Future iterations of this project could include a more detailed fuzzy rule set which covers different scenarios of various legs walking over objects. This addition would allow the robot to walk over more realistic surface typologies with minimal instability. In addition to improvements of the fuzzy rule base, further development could be put into the input and output membership functions to handle objects of varying size. Including more membership functions for each of the inputs and outputs will increase the size of the fuzzy rule set and make the project more complicated but the robot will be able to handle varying sized objects better than it does as presented in this project. Additional improvements could be made to the design of

the robot model and movement system into a non-rigid body which can shift its centre of mass. This dynamic stability system would improve the robots stability and could possibly be implemented with a separate fuzzy controller. This addition would improve the robots stability by allowing it to adapt to more unpredictable surface features while in motion. The limited scope of this project meant that the robot could only perform forward movement. Additional developments into movement algorithms could be made to allow the robot to steer itself or walk backwards. Lastly the kinematic model of the robot could be improved by implementing a [Proportional Integral Derivative \(PID\)](#) controller instead of the desired angle based control that was presented in this project. The [PID](#) controller would offer more accurate motor positioning with greater control of the smoothness of the motors which will improve the stability of the robot. Focusing on these changes could improve the robots stability while traversing over objects as well as the power needed to run the robot by limiting motor fatigue. Further simulations and analysis would need to be conducted to determine if these changes help the designed system perform better.

## 5.2 Conclusion

Through the design and development of simulation environments to represent the system, this project was able to successfully implement a fuzzy logic controlled 4-leg robot. In addition to implementing the project in a simulation environment, various parameters were measured and analysed to confirm that fuzzy logic control is a viable solution to control a robots movements when travelling on an uneven surface. Direct comparisons between fixed programmed movement and fuzzy logic controlled movement design show the effectiveness of fuzzy logic in this domain for improved stability and limiting motor fatigue. The fuzzy logic controlled 4-leg robot had improved stability when compared to a

fixed programmed movement design. Overall, the development of this project allowed for successful completion of all of the objectives outlined in Section [1.2](#).

# APPENDICES

# Appendix A

## MATLAB Code

```
1  % Constants
2  OBSTACLE_HEIGHT = 0.10; % m
3  WIDTH = 5; % m
4  LENGTH = 5; % m
5  RESOLUTION = 0.1; % m
6
7  % Define the plane
8  ground_x = -WIDTH:RESOLUTION:WIDTH; % in m
9  ground_y = -WIDTH:RESOLUTION:WIDTH; % in m
10 ground_z = zeros(size(ground_x, 2), size(ground_y, 2)); % N x M (101 x 101)
11
12 % Set Obstacle
13 obstacle_size = OBSTACLE_HEIGHT;
14 obstacle_pos = "R"; % Left or right of robot
15
16 if obstacle_pos == "L"
17     ground_z(find(string(ground_x) == "-1.7"):find(string(ground_x) == "-1.3"), ...
18             find(string(ground_y) == "-1"):find(string(ground_y) == "0"))) = obstacle_size;
19 elseif obstacle_pos == "R"
20     ground_z(find(string(ground_x) == "-1.5"):find(string(ground_x) == "-1.1"), ...
21             find(string(ground_y) == "0"):find(string(ground_y) == "1"))) = obstacle_size;
22 end
```

Figure A.1: init-ground.m.

```

1  function stop = stopLegsController(state, contact, prev_stop)
2
3  % Legs are not stopped by default
4  stop = [0; 0];
5  % Element 1 = front legs
6  % Element 2 = rear legs
7
8  % Downstroke is on the 3rd and 6th state from the body controller FSM
9
10 switch state
11     case {3, 6}
12         % Down stroke front legs
13         if prev_stop(1) ~= 1
14             % If leg is not already stopped, check if it should be stopped
15             if contact(1)
16                 prev_stop(1) = 1;
17                 stop(1) = prev_stop(1);
18             else
19                 prev_stop(1) = 0;
20                 stop(1) = prev_stop(1);
21             end
22         else
23             % If leg is already stopped, keep it stopped while in this state
24             prev_stop(1) = 1;
25             stop(1) = prev_stop(1);
26         end
27
28         % Down stroke rear legs
29         if prev_stop(2) ~= 1
30             % If leg is not already stopped, check if it should be stopped
31             if contact(2)
32                 prev_stop(2) = 1;
33                 stop(2) = prev_stop(2);
34             else
35                 prev_stop(2) = 0;
36                 stop(2) = prev_stop(2);
37             end
38         else
39             % If leg is already stopped, keep it stopped while in this state
40             prev_stop(2) = 1;
41             stop(2) = prev_stop(2);
42         end
43     otherwise
44         % Legs are not stopped by default
45         stop = [0; 0];
46 end

```

Figure A.2: stop-legs-controller.m.

```

1 function [a_next, b_next, stopped_a1_next, stopped_a2_next, state_next] = bodyControl(a, b, stopped_a1, stopped_a2, state)
2
3 ENABLE_FIS = 1;
4 fisFilename = "QuadRobotFIS_8";
5
6 a_next = [a(1); a(2)]; % Alpha angles
7 b_next = b; % Beta angles
8
9 % Set next stopped position to current stopped position
10 stopped_a1_next = stopped_a1;
11 stopped_a2_next = stopped_a2;
12
13 switch state
14 case 0
15     % Initial
16     a_next = [0; 0];
17     b_next = -20;
18     state_next = 1;
19 case 1
20     stopped_a1_next = a(1);
21     stopped_a2_next = a(2);
22     % FR/RL up
23     if ENABLE_FIS
24         fis = getFISCodeGenerationData(fisFilename);
25         output = evalfis(fis, [a(1) a(2) b]);
26         a_next = [output(1); output(2)];
27     else
28         a_next = [20; -20];
29     end
30     state_next = 2;
31 case 2
32     % FR/RL forward
33     b_next = 20;
34     state_next = 3;
35 case 3
36     % All down
37     a_next = [stopped_a1; stopped_a2];
38     %a_next = [0; 0];
39     state_next = 4;
40 case 4
41     % FL/RR up
42     stopped_a1_next = a(1);
43     stopped_a2_next = a(2);
44     if ENABLE_FIS
45         fis = getFISCodeGenerationData(fisFilename);
46         output = evalfis(fis, [a(1) a(2) b]);
47         a_next = [output(1); output(2)];
48     else
49         a_next = [-20; 20];
50     end
51     state_next = 5;
52 case 5
53     % FL/RR forward
54     b_next = -20;
55     state_next = 6;
56 case 6
57     % All down
58     a_next = [stopped_a1; stopped_a2];
59     %a_next = [0; 0];
60     state_next = 1;
61 otherwise
62     state_next = 0;
63 end

```

Figure A.3: body-controller.m.

# Appendix B

## Results Data

### B.1 Level Surface

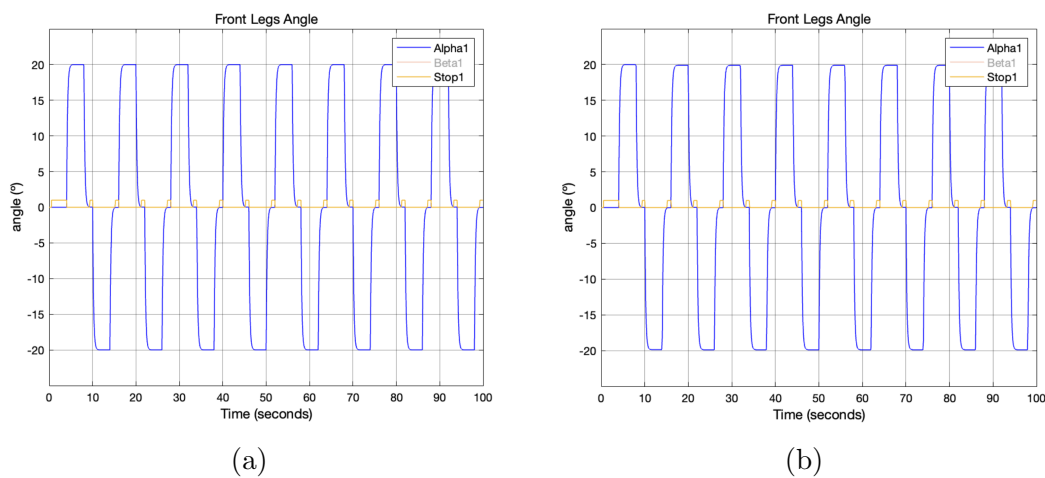
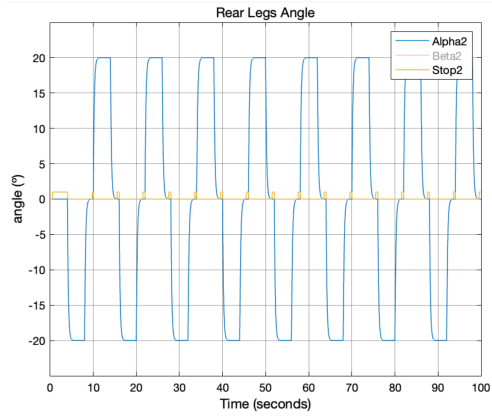
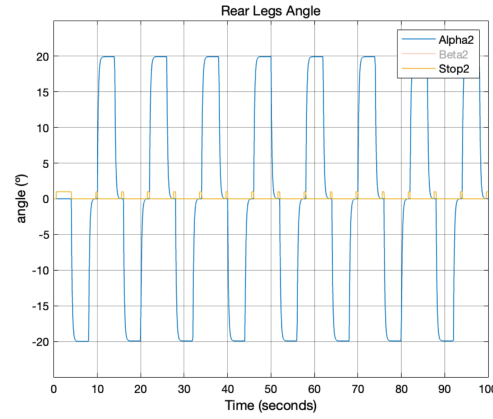


Figure B.1: Front legs angle on level surface (a) Fixed programmed control, (b) Fuzzy logic control.

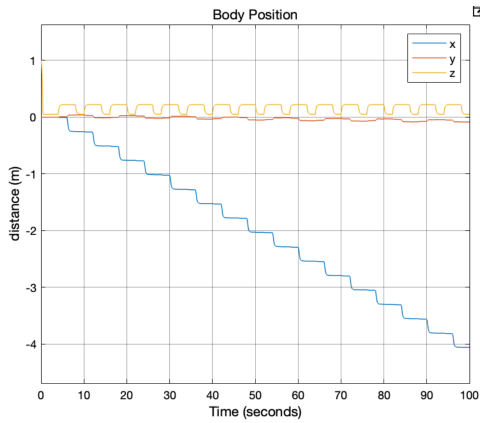


(a)

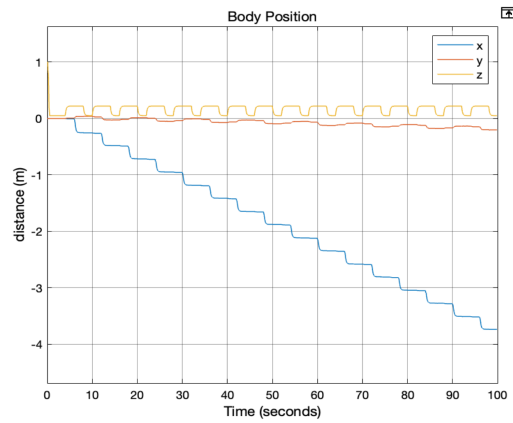


(b)

Figure B.2: Rear legs angle on level surface (a) Fixed programmed control, (b) Fuzzy logic control.

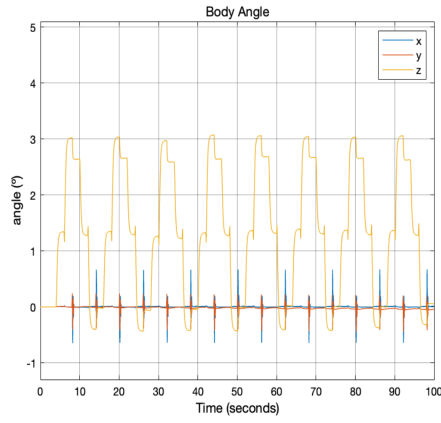


(a)

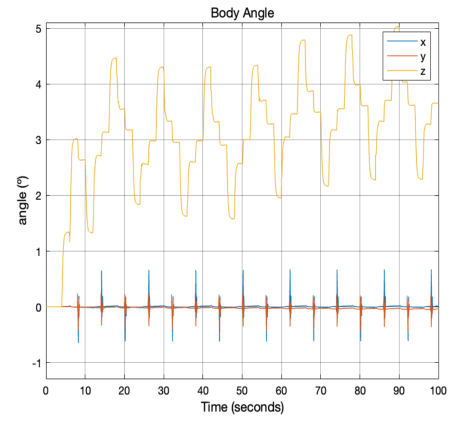


(b)

Figure B.3: Body position on level surface (a) Fixed programmed control, (b) Fuzzy logic control.

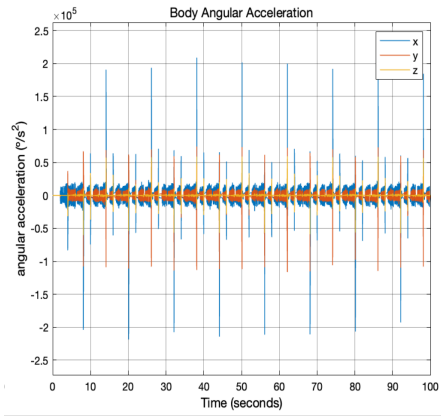


(a)

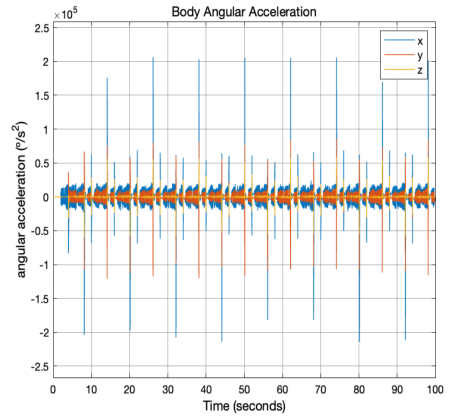


(b)

Figure B.4: Body angle on level surface (a) Fixed programmed control, (b) Fuzzy logic control.

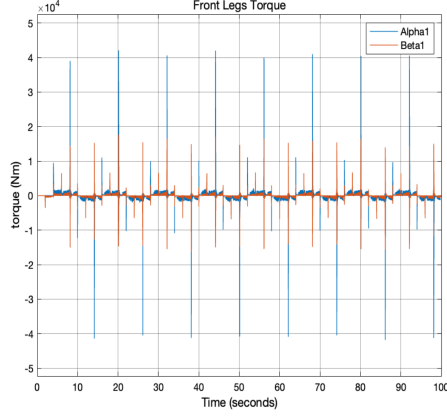


(a)

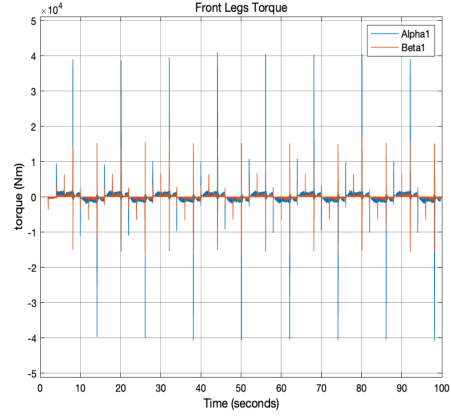


(b)

Figure B.5: Body angular acceleration on level surface (a) Fixed programmed control, (b) Fuzzy logic control.

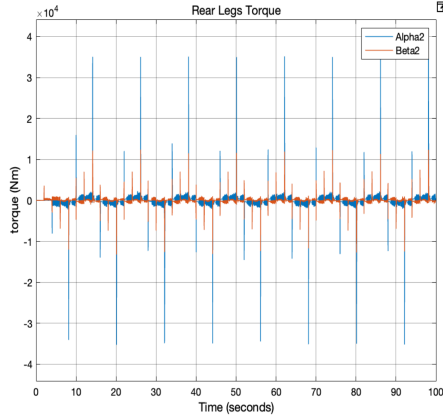


(a)

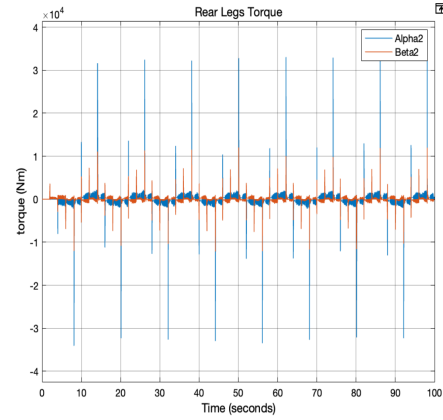


(b)

Figure B.6: Front legs torque on level surface (a) Fixed programmed control, (b) Fuzzy logic control.



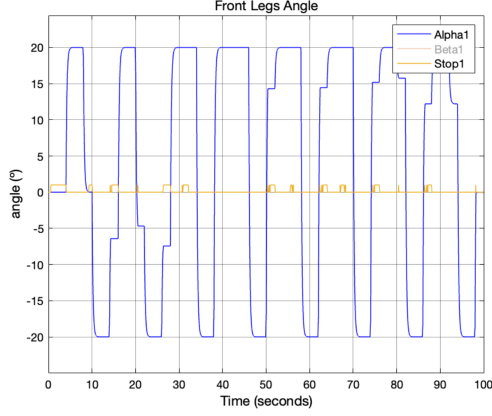
(a)



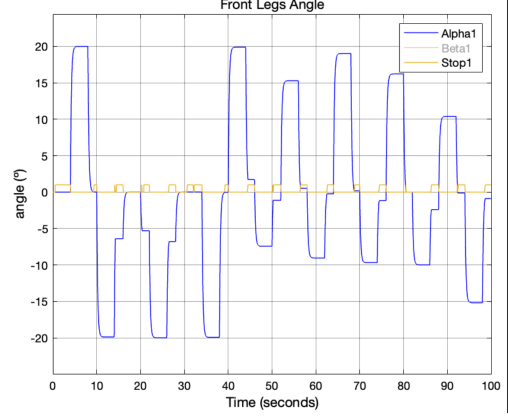
(b)

Figure B.7: Rear legs torque on level surface (a) Fixed programmed control, (b) Fuzzy logic control.

## B.2 Object Under Left Legs

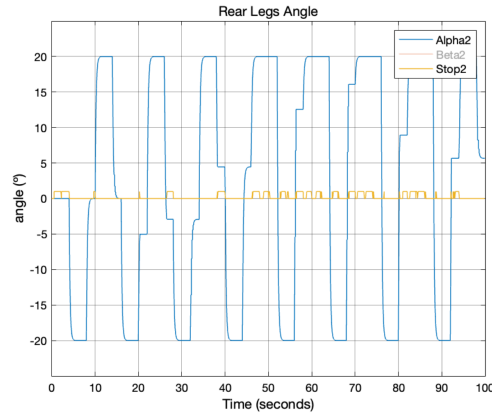


(a)

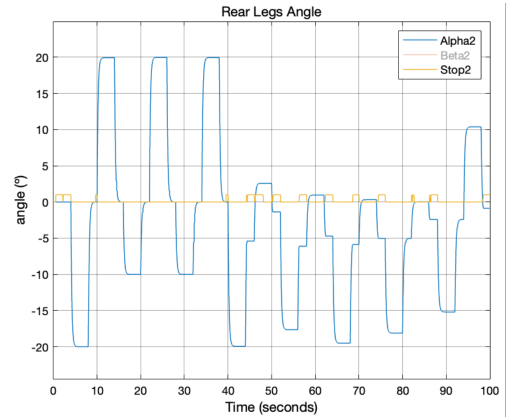


(b)

Figure B.8: Front legs angle walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.

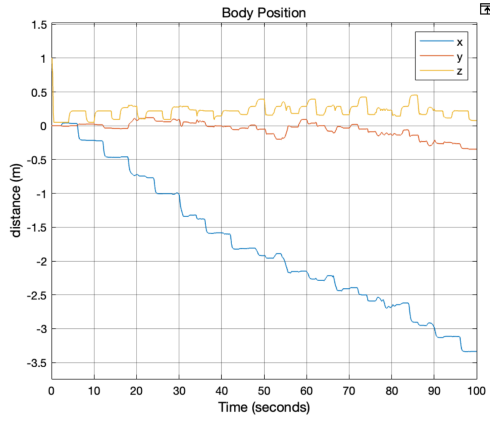


(a)

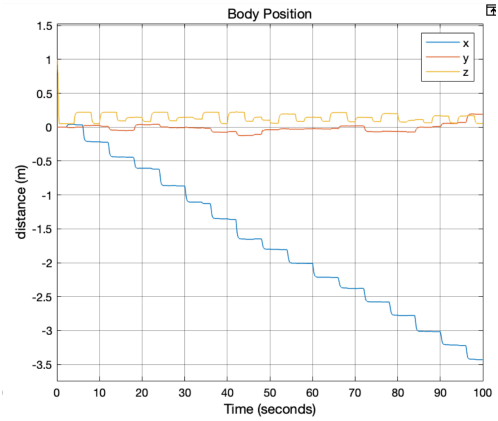


(b)

Figure B.9: Rear legs angle walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.

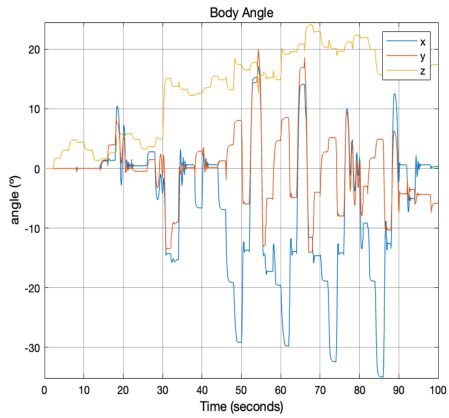


(a)

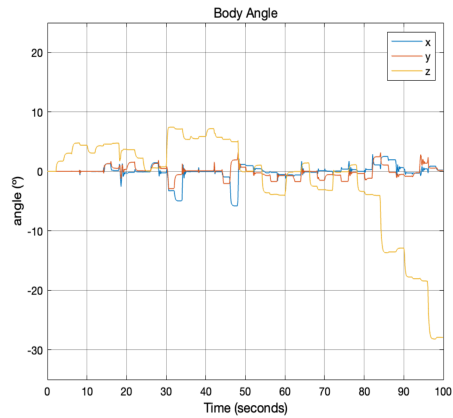


(b)

Figure B.10: Body position walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.

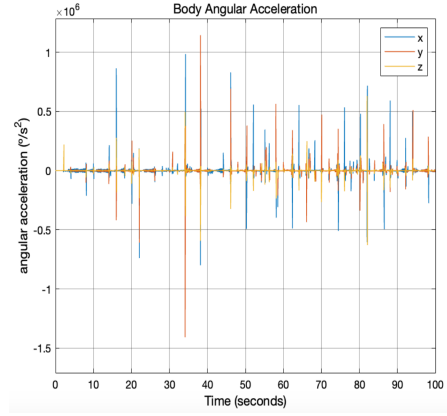


(a)

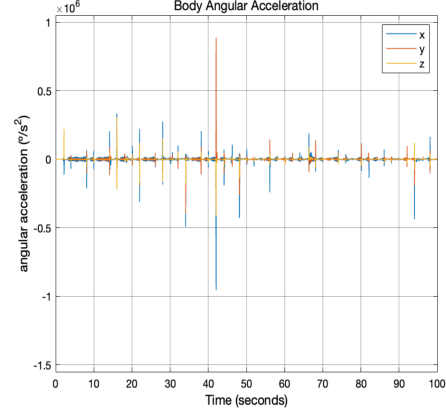


(b)

Figure B.11: Body angle walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.

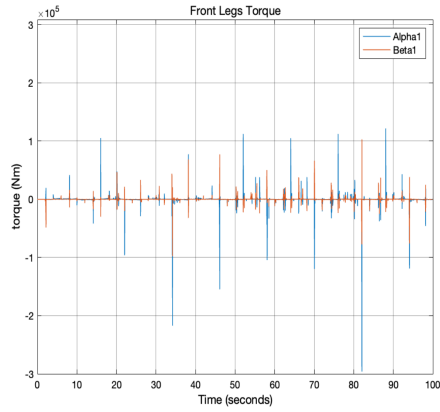


(a)

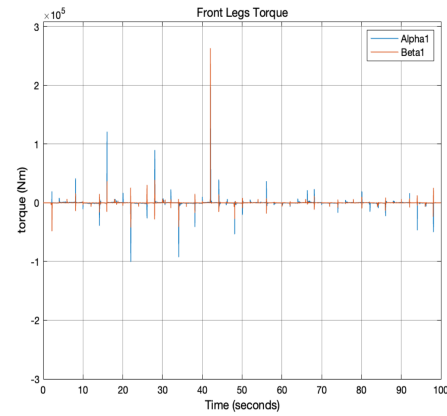


(b)

Figure B.12: Body angular acceleration walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.

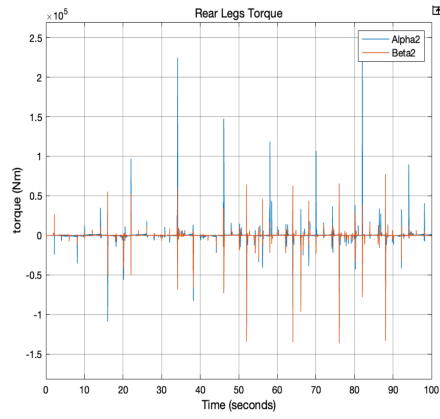


(a)

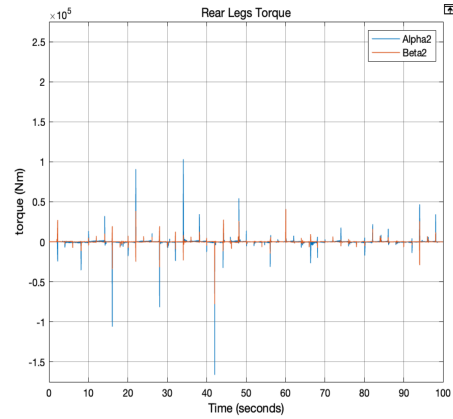


(b)

Figure B.13: Front legs torque walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.



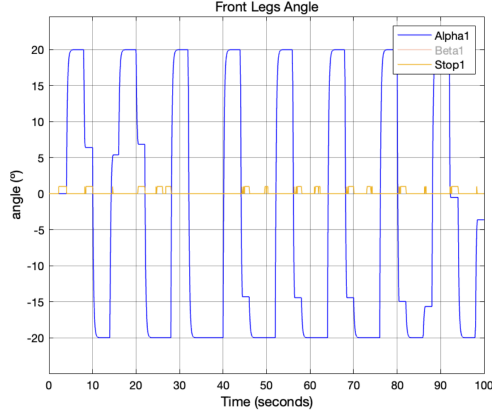
(a)



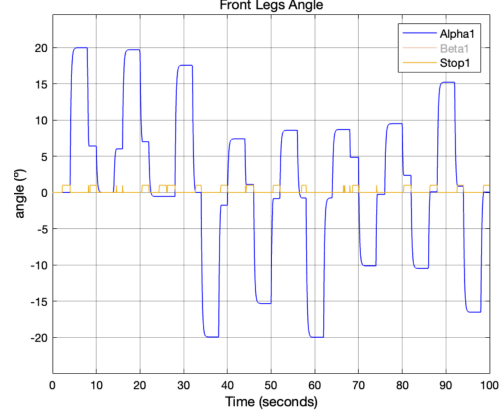
(b)

Figure B.14: Rear legs torque walking over object on the left (a) Fixed programmed control, (b) Fuzzy logic control.

### B.3 Object Under Right Legs

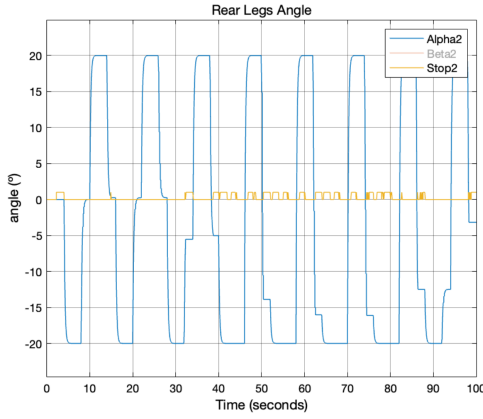


(a)

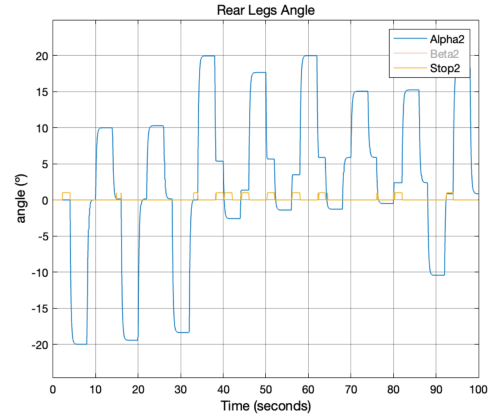


(b)

Figure B.15: Front legs angle walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.

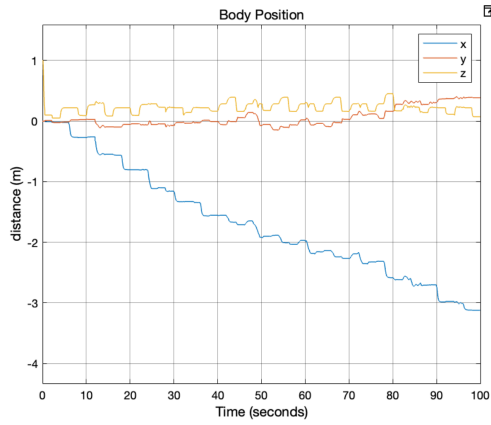


(a)

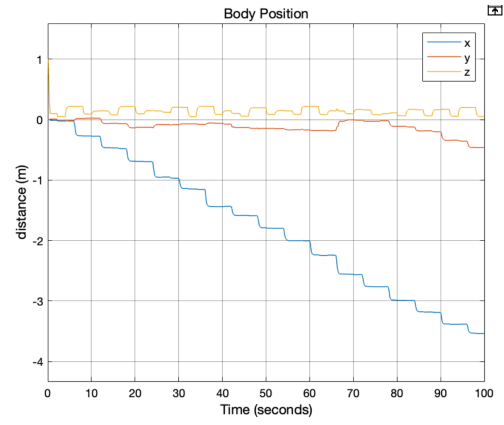


(b)

Figure B.16: Rear legs angle walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.

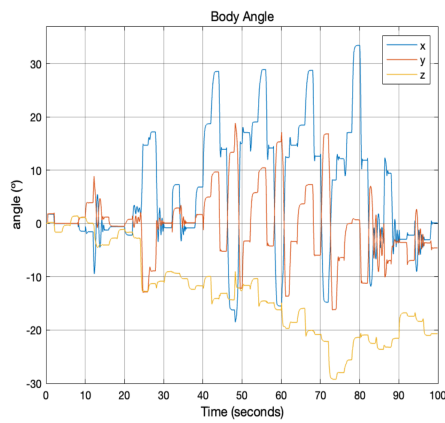


(a)

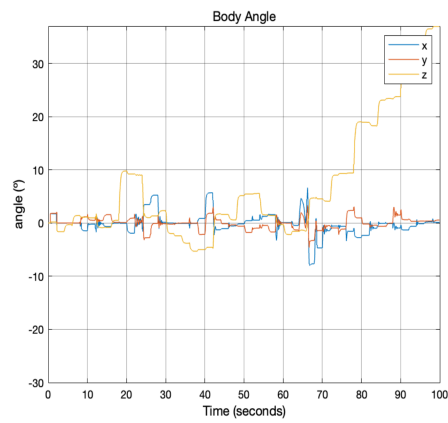


(b)

Figure B.17: Body position walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.

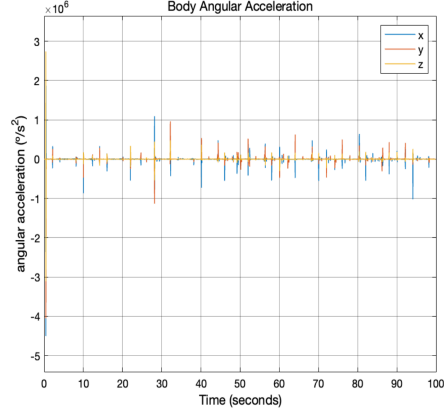


(a)

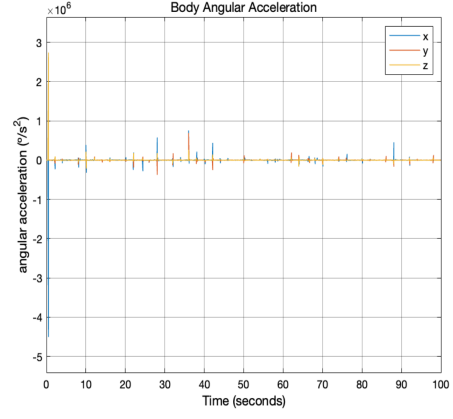


(b)

Figure B.18: Body angle walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.

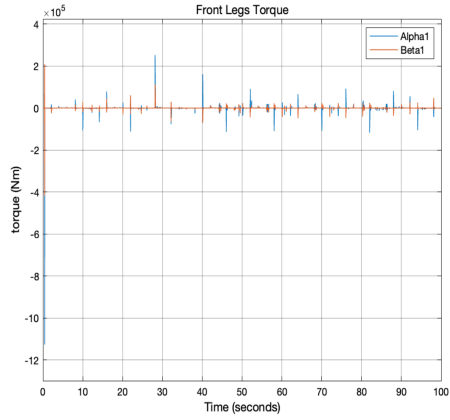


(a)

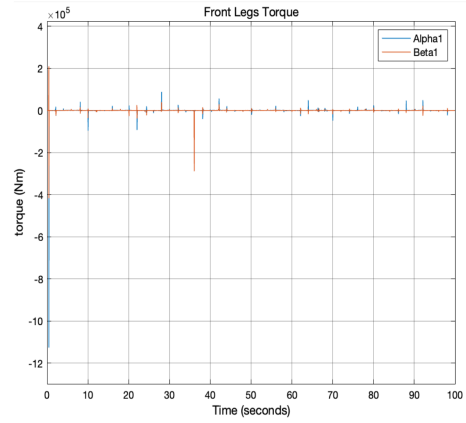


(b)

Figure B.19: Body angular acceleration walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.

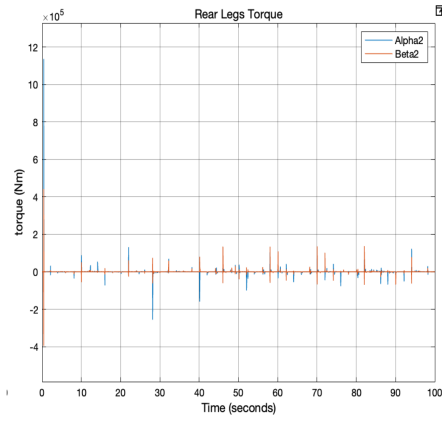


(a)

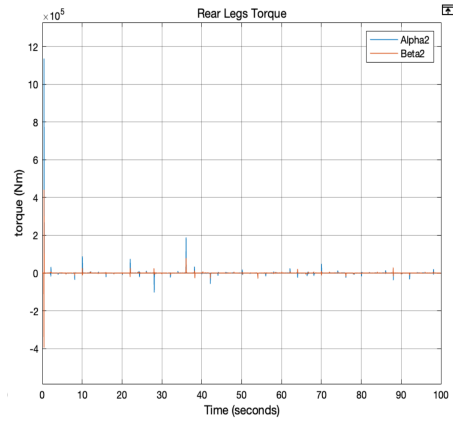


(b)

Figure B.20: Front legs torque walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.



(a)



(b)

Figure B.21: Rear legs torque walking over object on the right (a) Fixed programmed control, (b) Fuzzy logic control.

# References

- [1] Z. Sakr and E. M. Petriu, “Hexapod robot locomotion using a fuzzy controller,” in *2007 International Workshop on Robotic and Sensors Environments*, pp. 1–5, 2007.
- [2] C. Jayarathna, J. Wijayakulasooriya, and S. Kodituwakku, “Fuzzy logic and neural network control systems for backing up a truck and a trailer,” 06 2011.
- [3] R. J. Full, C. T. Farley, and J. M. Winters, *Musculoskeletal Dynamics in Rhythmic Systems: A Comparative Approach to Legged Locomotion*, pp. 192–205. New York, NY: Springer New York, 2000.
- [4] A. Globisz and D. Krawczyk, “Statically stable gait synthesis algorithm for 4-legged walking robot based on the ground reaction forces determination,” in *2018 19th International Conference on Research and Education in Mechatronics (REM)*, pp. 81–86, 2018.
- [5] H. Ishihara and K. Kuroi, “A four-leg locomotion robot for heavy load transportation,” in *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5731–5736, 2006.
- [6] F. Tracuzzi Spadaro and G. Gerardo Muscolo, *Modelling and Simulation of a Four Legged Robot*. 2020.

- [7] D. Neculescu and R. Jassemi-Zargani, “Extended kalman filter based sensor fusion for operational space control of a robot arm,” in *IMTC 2001. Proceedings of the 18th IEEE Instrumentation and Measurement Technology Conference. Rediscovering Measurement in the Age of Informatics (Cat. No.01CH 37188)*, vol. 2, pp. 915–918 vol.2, 2001.
- [8] B. Muntean, M. Zaha, and S. Grigorescu, “Analytic inverse kinematics model and trajectory planning for an 18 dof quadruped robot,” in *2023 27th International Conference on System Theory, Control and Computing (ICSTCC)*, pp. 277–282, 2023.
- [9] P. B, T. K, R. K. M, Y. S, R. P. J, and P. B, “Design and development of a quadruped robot for achieving multi-terrain locomotion,” in *2023 Second International Conference on Trends in Electrical, Electronics, and Computer Engineering (TEECCON)*, pp. 51–56, 2023.
- [10] M. Kern and P.-Y. Woo, “Implementation of a hexapod mobile robot with a fuzzy controller,” *Robotica*, vol. 23, no. 6, p. 681–688, 2005.
- [11] M. Chang, K. Kim, and D. Jeon, “Research on terrain identification of the smart prosthetic ankle by fuzzy logic,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 27, no. 9, pp. 1801–1809, 2019.
- [12] F. Zha, Q. Wang, P. Zheng, C. Chen, and W. Guo, “Research on dynamic stability prediction of large heavy six-legged robots on slope,” in *2018 11th International Workshop on Human Friendly Robotics (HFR)*, pp. 7–12, 2018.